The overarrows package*

Julien Labbé
https://github.com/julienlabbe/latex-packages

October 19, 2025

Abstract

A LaTeX package to create custom arrows over math expressions, mainly for vectors (but arrows can as well be drawn below). Arrows stretch with content, scale with math styles, and have a correct kerning when a subscript follows.

Short example:

```
\NewOverArrowCommand{\overrightharpoon}{%
  end=\rightharpoonup
}

\begin{align*}
  &\overrightharpoon{v} && \overrightharpoon{v}_{subscript} \\
  &\overrightharpoon{ABCD} && \overrightharpoon*{v}_{subscript}
\end{align*}
```

$$\overrightharpoon{v} \qquad \overrightharpoon{v}_{subscript}$$
$$\overrightharpoon{ABCD} \qquad \overrightharpoon*{v}_{subscript}$$

Predefined commands are also provided:

- to typeset vectors:
$$\vec{v} \qquad \overrightarrow{AB},$$
- to draw arrows of various shapes above math expressions:
$$\overrightarrow{AB} \quad \overleftarrow{AB} \quad \overleftrightarrow{AB} \quad \overrightarrow{AB} \quad \overleftarrow{AB} \quad \overrightarrow{AB} \quad \overrightarrow{AB} \quad \overrightarrow{AB},$$
- to draw arrows of various shapes under math expressions:
$$\underrightarrow{AB} \quad \underleftarrow{AB} \quad \underleftrightarrow{AB} \quad \underrightarrow{AB} \quad \underrightarrow{AB} \quad \underrightarrow{AB} \quad \underrightarrow{AB} \quad \underrightarrow{AB}.$$

*This document corresponds to overarrows v1.5, dated 2025/10/19.

Contents

1 Presentation of the package **4**

2 Introduction **4**
2.1 Vector arrows . . . 4
2.2 Stack and arrow macros . . . 5
2.3 Extensible arrows . . . 5

3 Quick start **6**
3.1 Loading the package overarrows . . . 6
3.2 Commands creation . . . 6
3.3 Start and end of the arrow . . . 6
3.4 Size and position of the arrow . . . 8
3.5 Symbols assemblage . . . 9
3.6 Drawing the arrow with TikZ . . . 11
3.7 Drawing the arrow with PSTricks . . . 12
3.8 Drawing the arrow with LaTeX picture environment . . . 12

4 User interface **13**
4.1 Package options . . . 13
4.1.1 esvect configuration . . . 13
4.1.2 Predefined commands . . . 14
4.1.3 Other options . . . 16
4.2 Commands . . . 17
4.2.1 Macro for commands creation . . . 17
4.2.2 Useful macros for symbols assemblage . . . 18
4.2.3 Useful lengths for TikZ, PSTricks or `picture` environment . . . 20
4.2.4 Vectors macros . . . 21
4.2.5 Predefined commands . . . 21
4.3 Keys . . . 23
4.3.1 Arrow position and length settings . . . 23
4.3.2 Subscripts detection setting . . . 25
4.3.3 Symbols assemblage settings . . . 26
4.3.4 TikZ settings . . . 27
4.3.5 PSTricks settings . . . 29
4.3.6 Picture environment settings . . . 30
4.4 Advanced commands and keys . . . 30
4.4.1 Advanced commands . . . 31
4.4.2 Advanced keys . . . 31

5 Complements **32**
5.1 Know issues . . . 32
5.1.1 Math font change . . . 32
5.1.2 Detection of non standard subscripts . . . 32
5.2 Package dependencies . . . 33
5.3 Alternatives . . . 33
5.4 Changelog . . . 34

6 Implementation **34**
6.1 Management of options . . . 34
6.1.1 Declaration of conditionals . . . 34
6.1.2 Declaration of options . . . 35
6.1.3 Options processing . . . 36
6.2 Package dependencies . . . 36
6.3 Configuration of subscripts detection . . . 37
6.4 Management of keys . . . 38
6.4.1 Family declaration and setters . . . 38
6.4.2 Common keys . . . 38
6.4.3 Keys for the `symb` method . . . 40
6.4.4 Keys for the `tikz` method . . . 42
6.4.5 Keys for the `pstricks` method . . . 43
6.4.6 Keys for the `picture` method . . . 43
6.5 Commands . . . 44
6.5.1 Macros for symbols assemblage . . . 44
6.5.2 Macros for fixed length arrows . . . 45
6.5.3 Stack macros . . . 46
6.5.4 Macro for commands creation . . . 48
6.5.5 Starred variant . . . 49
6.5.6 `\vv` vector command . . . 50
6.5.7 Predefined commands . . . 50
6.5.8 Test macros . . . 52

Index **54**

1 Presentation of the package

The overarrows package allows to create commands for drawing arrows over math expressions. These arrows:

- are fully customisable, at command definition, through a key-value interface;
- stretch with the content and can cover many characters, like in $\overrightarrow{AB}$;
- scale with math styles[1], like in $\vec{v}_{\vec{u}_{\vec{w}}}$.

Commands created with the overarrows package are provided with a starred variant, that removes the extra end space generated by the arrow. This is particularly useful when the command is followed by a subscript. For example, the velocity of the center of mass can be written with exactly the same kerning when scalar v_{cm} or vector $\vec{v}_{\mathrm{cm}}$ (no extra space before the subscript, unlike the output of the unstarred variant: $\vec{v}\,_{\mathrm{cm}}$).

The overarrows package was primitively written for vectors, but in a highly customisable way. It can be used to define a large variety of arrows, using math symbols, or drawing commands from PGF/TikZ or PSTricks. It's also possible to create commands that draw the arrows under. Some predefined commands are provided, giving[2], for arrow over:

$$\overrightarrow{\alpha+\beta}\quad \overleftarrow{\alpha+\beta}\quad \overleftrightarrow{\alpha+\beta}\quad \overrightarrow{\alpha+\beta}\quad \overleftarrow{\alpha+\beta}\quad \overrightarrow{\alpha+\beta}\quad \overleftarrow{\alpha+\beta}\quad \overline{\alpha+\beta}$$

and for arrow under :

$$\underrightarrow{\alpha+\beta}\quad \underleftarrow{\alpha+\beta}\quad \underleftrightarrow{\alpha+\beta}\quad \underrightarrow{\alpha+\beta}\quad \underleftarrow{\alpha+\beta}\quad \underrightarrow{\alpha+\beta}\quad \underleftarrow{\alpha+\beta}\quad \underline{\alpha+\beta}.$$

2 Introduction

2.1 Vector arrows

Vectors are commonly typeset in bold face, or with an arrow above[3]. For this second convention, TEX/LATEXprovides the command `\vec`, which accents its content (using the `\mathaccent` command) with the character → (`\mathchar"017E` in Computer Modern font). But → isn't extensible[4], and gives: $\vec{v}$, $\vec{AB}$ or $\vec{\mathrm{grad}}$ (there's no command `\widevec` analogous to `\widehat`).

An extensible alternative is given by the command `\overrightarrow`, available in TEX/LATEX, and which is redefined by the commonly used amsmath package. But its arrow, built with the `\rightarrow` symbol →, is too large, using the default *Computer Modern* font: $\overrightarrow{AB}$. Another alternative is the esvect package, which provides the `\vv` command and a set of custom arrows: $\overrightarrow{AB}$, $\overrightarrow{AB}$, $\overrightarrow{AB}$, $\overrightarrow{AB}$, $\overrightarrow{AB}$, $\overrightarrow{AB}$, $\overrightarrow{AB}$, $\overrightarrow{AB}$.

[1] `\displaystyle`, `\textstyle`, `\scriptstyle` and `\scriptscriptstyle`.

[2] Displayed here with the `old-arrows` option.

[3] See, for example: International Organization for Standardization. (2019). *Quantities and units – Part 2: Mathematics* (ISO Standard No. 80000-2:2019). `https://www.iso.org/standard/64973.html`.

[4] In fact, with the unicode engines LuaTEX and XETEX, the command `\Umathaccent` can now define extensible accents. This is used by the unicode-math package, which also set the arrows displayed by `\vec` and `\overrightarrow` in a coherent manner.

2.2 Stack and arrow macros

It's worth looking at the definition of amsmath `\overrightarrow` command:

```
\long macro:->\mathpalette {\overarrow@ \rightarrowfill@ }
```

Three macros are used here:

`\mathpalette` adapts the output to the current math style;

`\overarrow@` is the *stack macro*, that puts the arrow above the content;

`\rightarrowfill@` is the *arrow macro*, that holds the content of the arrow.

The command `\vv` from esvec is defined with a very similar way, using its own stack macro (`\overvect@`) and arrow macro (`\vectfill@`).

The overarrows package uses the same mechanism. Arrow and stack macros are set, at command creation, through a key-value interface provided by the pgfkeys package (after creation, however, the command definition is static and the key-value interface is not used).

2.3 Extensible arrows

Arrows drawn by the commands `\overrightarrow` or `\vv` are built by joining math symbols, and made extensible by repetition of the central symbol[5]. Thus, the line of the macro `\overrightarrow` is made by repetition of command `\relbar` — (which simply corresponds to the minus sign), while `\vv` use its own command `\relbareda` -.

This method may generate some undesirable spacing issues, when symbols badly overlap. See, for example, the output of amsmath `\overrightarrow` (left) and esvect `\vv` (right) in `\scriptscriptstyle` math style (scaled by a factor 4):

$$\overrightarrow{long\ vector} \qquad \overrightarrow{long\ vector}.$$

While the arrow on the left lets guess where the symbols — overlap, the arrow on the right present unwanted spaces and show clearly its composition as association of the symbols –, - and →.

By default, the overarrows package uses the same mechanism to extend arrows according to their contents. Settings and tools are provided to perform fine tuning and avoid spacing issues. As example, see below the `\overrightarrow` and `\vv` commands, as redefined by overarrows (in `\scriptscriptstyle` and scaled by a factor 4):

$$\overrightarrow{long\ vector} \qquad \overrightarrow{long\ vector}$$

The overarrows package also provides an alternative mechanism. When used, the length `\overarrowlength` is set, according to the arrow command content, and can be employed, for example, to draw arrows using PGF/TikZ, PSTricks or the LaTeX picture environment.

[5] Using the TeX `\cleaders` command.

3 Quick start

3.1 Loading the package overarrows

To load the overarrows, simply add in preamble, before the "\begin{document}":

```
\usepackage{overarrows}
```

Options can be given, in a comma-separated list. For example, to use the predefined commands shown in the section 1, page 4, write:

```
\usepackage[allcommands, old-arrows]{overarrows}
```

This define the commands (described in section 4.2.5, page 21):

- \overrightarrow → P. 21
- \overleftarrow → P. 22
- \overleftrightarrow → P. 22
- \overrightharpoonup → P. 22
- \overrightharpoondown → P. 22
- \overleftharpoonup → P. 22
- \overleftharpoondown → P. 22
- \overbar → P. 22
- \underrightarrow → P. 22
- \underleftarrow → P. 22
- \underleftrightarrow → P. 22
- \underrightharpoonup → P. 23
- \underrightharpoondown → P. 23
- \underleftharpoonup → P. 23
- \underleftharpoondown → P. 23
- \underbar → P. 23

Note that the old-arrows → P. 16 option may give bad results, if math fonts have been changed. Simply remove the option in this case.

Many other options are available. See the complete list, page 13.

3.2 Commands creation

Commands are created with \NewOverArrowCommand → P. 17. This macro take two mandatory arguments : the name of the command and the arrow configuration as comma-separated list of key-values. By default, a right arrow is set:

```
\NewOverArrowCommand{\myovercmd}{}
$\myovercmd{test}$
```

$\overrightarrow{test}$

Commands are defined with a starred variant, designed to handle subscripts:

```
$ v_{sub} \qquad \myovercmd{v}_{sub} \qquad \myovercmd*{v}_{sub} $
```

$v_{sub} \qquad \overrightarrow{v}_{sub} \qquad \overrightarrow{v_{sub}}$

3.3 Start and end of the arrow

Extremities of the arrow are set by the keys start → P. 26 and end → P. 26. For example, an arrow starting with a hook (symbols \lhook ↪) and ending with two heads (symbol \twoheadrightarrow ↠) is defined by:

```
\NewOverArrowCommand{\overhooktwoheadrightarrow}{%
  start=\lhook, end=\twoheadrightarrow,
}
```

Note that `\twoheadrightarrow` must be defined, as it is not in LaTeX. This can be done with the package amssymb, by adding in preamble:

```
\usepackage{amssymb}
```

But with the previous definition, the result of the command `\overhooktwoheadrightarrow` is faulty:

```
$ \overhooktwoheadrightarrow{v} \qquad \overhooktwoheadrightarrow{AB} $
```

$$\overhooktwoheadrightarrow{v} \qquad \overhooktwoheadrightarrow{AB}$$

The problem comes from symbols junction and the trimming used to obtain their overlap. It can be solved with the keys `trim start` → P. 26 and `trim end` → P. 26, which are numbers and set the corresponding trimming in math units (typically `1/18 em`). Appropriate values gives better results:

```
\NewOverArrowCommand{\overhooktwoheadrightarrow}{%
  start=\lhook, end=\twoheadrightarrow,
  trim start=1.5, trim end=2,
}
$ \overhooktwoheadrightarrow{v} \qquad \overhooktwoheadrightarrow{AB} $
```

$$\overhooktwoheadrightarrow{v} \qquad \overhooktwoheadrightarrow{AB}$$

If the math font differs from the default *Computer Modern*, the central part of the arrow may have inappropriate position or line width. This is because the default symbol used for the arrow line is `\relbareda` - from the esvect package[6]. If needed, try to set the `middle` → P. 26 key with the symbol `\relbar` —. The trimming should also be adapted:

```
\NewOverArrowCommand{\overhooktwoheadrightarrow}{%
  start=\lhook, end=\twoheadrightarrow, middle=\relbar,
  trim start=0, trim end=3, trim middle=5,
}
$ \overhooktwoheadrightarrow{v} \qquad \overhooktwoheadrightarrow{AB} $
```

$$\overhooktwoheadrightarrow{v} \qquad \overhooktwoheadrightarrow{AB}$$

Finding the correct values for `trim start` → P. 26, `trim end` → P. 26 and `trim middle` → P. 26 may need many trials. For this purpose, the macro `\TestOverArrow` → P. 18 displays the result of a command for different lengths and math styles:

[6] Except if the unicode-math package is used with a math font that provides the `\harrowextender` symbol (see the `middle config=auto` key).

```
\TestOverArrow{\overhooktwoheadrightarrow}
```

\displaystyle	\textstyle	\scriptstyle	\scriptscriptstyle
$\overhooktwoheadrightarrow{v}$	$\overhooktwoheadrightarrow{v}$	$\overhooktwoheadrightarrow{v}$	$\overhooktwoheadrightarrow{v}$
$\overhooktwoheadrightarrow{AB}$	$\overhooktwoheadrightarrow{AB}$	$\overhooktwoheadrightarrow{AB}$	$\overhooktwoheadrightarrow{AB}$
$\overhooktwoheadrightarrow{\text{grad}}$	$\overhooktwoheadrightarrow{\text{grad}}$	$\overhooktwoheadrightarrow{\text{grad}}$	$\overhooktwoheadrightarrow{\text{grad}}$
$\overhooktwoheadrightarrow{my\ long\ vector}$	$\overhooktwoheadrightarrow{my\ long\ vector}$	$\overhooktwoheadrightarrow{my\ long\ vector}$	$\overhooktwoheadrightarrow{my\ long\ vector}$

3.4 Size and position of the arrow

A command `\OverRightarrow`, built with the symbols `\Relbar` = and `\Rightarrow` ⇒, gives:

```
\NewOverArrowCommand{\OverRightarrow}{%
  start=\Relbar,
  middle=\Relbar,
  end=\Rightarrow,
  trim=4,
}
$ \OverRightarrow{v} \qquad \OverRightarrow{AB} $
```

$\overRightarrow{v} \qquad \overRightarrow{AB}$

The key `trim` → P. 26 sets `trim start` → P. 26, `trim middle` → P. 26 and `trim end` → P. 26 with the same value.

The previous arrow is visually too big. The macro `\smallermathstyle` → P. 19 allows to obtain a better result:

```
\NewOverArrowCommand{\OverRightarrow}{%
  start={\smallermathstyle\Relbar},
  middle={\smallermathstyle\Relbar},
  end=\Rightarrow,
  trim=4,
}
$ \OverRightarrow{v} \qquad \OverRightarrow{AB} $
```

$\overRightarrow{v} \qquad \overRightarrow{AB}$

Note that `\smallermathstyle` → P. 19 should not be used for `end` → P. 26, because this last is formatted with the same math style as `start` → P. 26.

It would be better to add an extra space between the arrow and the content of the command. This can be done with the key `space after arrow` → P. 25:

```
\NewOverArrowCommand{\OverRightarrow}{%
  start={\smallermathstyle\Relbar},
  middle={\smallermathstyle\Relbar},
  end=\Rightarrow,
  trim=4,
  space after arrow=0.25ex,
}
$ \OverRightarrow{v} \qquad \OverRightarrow{AB} $
```

$\overRightarrow{v} \qquad \overRightarrow{AB}$

Default arrows are slightly shifted to the right. For a left arrow, this should be reversed, using the keys `shift left`→P. 24 and `shift right`→P. 24. These keys set the corresponding shifts, in math units. Example:

```
\NewOverArrowCommand{\OverLeftarrow}{%
  start={\smallermathstyle\Leftarrow},
  middle={\smallermathstyle\Relbar},
  end=\Relbar,
  trim=4,
  space after arrow=0.25ex,
  shift left=0, shift right=2,
}
$ \OverLeftarrow{v} \qquad \OverLeftarrow{AB} $
```

$$\overleftarrow{v} \qquad \overleftarrow{AB}$$

Finally, the key `arrow under`→P. 24 places the arrow below the content, instead of above (and `space before arrow`→P. 25 sets the space upon it):

```
\NewOverArrowCommand{\UnderLeftRightarrow}{%
  start={\smallermathstyle\Leftarrow},
  middle={\smallermathstyle\Relbar},
  end=\Rightarrow,
  trim=4,
  arrow under,
  space before arrow=0.5ex,
  shift left=0, shift right=0,
}
$ \UnderLeftRightarrow{v} \qquad \UnderLeftRightarrow{AB} $
```

$$\underleftrightarrow{v} \qquad \underleftrightarrow{AB}$$

3.5 Symbols assemblage

Many LaTeX math symbols are built by assemblage, using the macro `\joinrel`[7] which remove 3 math units of horizontal space. The overarrows package provides a flexible version of `\joinrel`, called `\xjoinrel`→P. 19, which remove an arbitrary number of math units, given as optional argument.

Symbols association is then simple. As example, one can define a triple tail macro `\tttail` from the symbol `\succ` $\succ$:

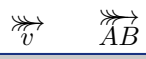
```
\newcommand*{\tttail}{\succ\xjoinrel[10]\succ\xjoinrel[10]\succ}
$ \tttail $
```

$$\succ\!\!\succ\!\!\succ$$

Thus defined, the macro `\tttail` can be used in arrow definition:

[7] For example, the symbol `\models` $\models$ is defined as `\mathrel{|}\joinrel\Relbar` and corresponds to the assemblage of a vertical line $|$ and the symbol `\Relbar` $=$. The command `\mathrel` modifies the spacing according to the math relation class ; `\Relbar` corresponds to the equal sign (it's definition is `\mathrel{=}`).

```
\NewOverArrowCommand{\overttttailrightarrow}{%
  start={\tttail},
  end={\rightarrow},
  trim start=12,
  shift left=0, shift right=0,
  space after arrow=.2ex,
  min length=24,
}
$ \overttttailrightarrow{v} \qquad \overttttailrightarrow{AB} $
```



Here the `min length`→P. 23 key was added to ensure a minimum length (in math units) when the content of the command is small (as for a single character).

The previous arrow would be better with a smaller tail, and this can be done with the macro `\smallermathstyle`→P. 19. But a small tail and a normal sized head are not aligned; as `{\smallermathstyle\tttail}\xjoinrel[8]\rightarrow` gives:



The solution comes from the command `\vcenter` which centers materials on math axis. The tail must then be wrapped in a `\hbox`:

```
\NewOverArrowCommand{\overttttailrightarrow}{%
  start={\vcenter{\hbox{$\smallermathstyle\tttail$}}},
  end={\rightarrow},
  trim start=12,
  shift left=0, shift right=0,
  space after arrow=.2ex,
  min length=24,
}
$ \overttttailrightarrow{v} \qquad \overttttailrightarrow{AB} $
```



Text symbols, namely symbols that are not defined in math mode, can also be used. They should yet be enclosed in the `\text` macro, from the amsmath package, to be correctly displayed and correctly scaled according to math style. With, for example, the arrow heads given by the symbols 40 and 41 of the *lasy* font:

```
\newcommand*{\leftarrowhead}{\usefont{U}{lasy}{m}{n}\symbol{40}}
\newcommand*{\righttarrowhead}{\usefont{U}{lasy}{m}{n}\symbol{41}}
\NewOverArrowCommand{\overrightleftarrow}{%
  start=\text{\righttarrowhead},
  end=\text{\leftarrowhead},
  trim start=0.7,  trim end=0.7,
  min length=20,
  shift leftright=-2,
}
$ \overrightleftarrow{AB} \qquad \scriptstyle\overrightleftarrow{AB} $
```



3.6 Drawing the arrow with TikZ

In addition to the default method presented previously (assemblage of symbols, as described in section 2.3, page 5), the overarrows package has an alternative method to draw the arrow. This one allows the use of graphic languages such as PGF/TikZ.

Drawing arrows with TikZ requires to load the tikz package and its library `arrows.meta`. This can be simply done by passing the `tikz`^{→ P. 16} option to the overarrows package[8]:

```
\usepackage[tikz]{overarrows}
```

To use PGF/TikZ language, the optional argument `tikz` must be passed to `\NewOverArrowCommand`^{→ P. 17}. TikZ pictures are not extensible. That's why the overarrows package provides three lengths that can be used in TikZ commands:

- `\overarrowlength`^{→ P. 20} for the arrow length,
- `\overarrowthickness`^{→ P. 20} and `\overarrowsmallerthickness`^{→ P. 20} for the arrow thickness.

These lengths are computed at each utilisation of a command created with the `tikz` optional argument.

Without any other configuration, a right arrow is drawn:

```
\NewOverArrowCommand[tikz]{\overtikzarrow}{}
$ \overtikzarrow{v} \qquad \overtikzarrow{AB} $
```

$$\overrightarrow{v} \qquad \overrightarrow{AB}$$

Keys to use Tikz are described in section 4.3.4, page 27. Main keys are: `tikz options`^{→ P. 28}, `path options`^{→ P. 28} and `path`^{→ P. 28}. It's also possible to append settings with `add tikz options`^{→ P. 28} and `add path options`^{→ P. 28}. The full TikZ command used to draw the arrow can as well be entirely redefined with the key `tikz command`^{→ P. 28}

Here is a example of an arrow drawn with TikZ[9]:

```
\NewOverArrowCommand[tikz]{\overarchedleftrightarrow}{%
  add tikz options={y=\overarrowlength},
  add tikz options={line width={\overarrowsmallerthickness}},
  path options={arrows={<[scale=0.5]->[scale=0.5]}},
  path={(0,0) arc (-250:70:0.5 and 0.1)},
  center arrow,
  min length=25,
  space after arrow=0.4ex,
}
$ \overarchedleftrightarrow{v} \qquad \overarchedleftrightarrow{ABCD} $
```

$$\overleftrightarrow{v} \qquad \overleftrightarrow{ABCD}$$

[8] Note that the `tikz`^{→ P. 16} option isn't mandatory to use TikZ commands in overarrows. The tikz package and its library `arrows.meta` can be loaded independently.

[9] TikZ arrows are very powerfull, but much slower to draw than the default method using assemblage of math symbols.

3.7 Drawing the arrow with PSTricks

In addition to PGF/TikZ, the arrow can be drawn with PSTricks macros. For this, the optional argument `pstricks` must be passed to `\NewOverArrowCommand`→P. 17. Like with `tikz`, the three lengths `\overarrowlength`→P. 20, `\overarrowthickness`→P. 20 and `\overarrowsmallerthickness`→P. 20 can be used in PSTricks commands. By default, a right arrow is drawn:

```
\NewOverArrowCommand[pstricks]{\overpstarrow}{}
$ \overpstarrow{v} \qquad \overpstarrow{AB} $
```

$\vec{v} \qquad \overrightarrow{AB}$

The pstricks package has to be loaded (for example, using the `pstricks`→P. 16 option of the overarrows package)

Keys to use PSTricks commands are described in section 4.3.5, page 29. The main keys are `pstricks command`→P. 29, `psset`→P. 29, `arrow`→P. 29, `geometry`→P. 29 an `line thickness`→P. 29. Examples:

```
\NewOverArrowCommand[pstricks]{\overreddisks}{%
  psset={linecolor=red}, arrow=*-*, center arrow,
}
$ \overreddisks{v} \qquad \overreddisks{AB} $
```

$\vec{v} \qquad \overrightarrow{AB}$

```
\NewOverArrowCommand[pstricks]{\ellipticarrow}{%
  pstricks command={%
    \psellipticarcn{->}%^^A avoid space before coordinates
    (0.5\overarrowlength,0.2\overarrowlength)%^^A avoid space before coordinates
    (0.5\overarrowlength,0.2\overarrowlength)
    {170}{10}
  },
  geometry={(0,0.2\overarrowlength)(\overarrowlength,0.4\overarrowlength)},
  line thickness={\overarrowsmallerthickness},
  center arrow,
}
$ \ellipticarrow{v} \qquad \ellipticarrow{AB} $
```

$\vec{v} \qquad \overrightarrow{AB}$

3.8 Drawing the arrow with LaTeX picture environment

Without any other package, arrows can also be drawn with the LaTeX `picture` environment. In this case, the optional argument `picture` must be passed to `\NewOverArrowCommand`→P. 17. As with `tikz` or `pstricks`, the three lengths `\overarrowlength`→P. 20, `\overarrowthickness`→P. 20 and `\overarrowsmallerthickness`→P. 20 are available and can be used in `picture` drawing commands. By default, a right vector is drawn:

```
\NewOverArrowCommand[picture]{\overpictarrow}{}
$ \overpictarrow{v} \qquad \overpictarrow{AB} $
```

$\vec{v} \qquad \overrightarrow{AB}$

If overarrows is loaded with the option `pstarrows`→P. 16, the package pict2e is used and a PSTricks style vector arrows is set. This gives:

```
\NewOverArrowCommand[picture]{\overpictarrow}{}
$ \overpictarrow{v} \qquad \overpictarrow{AB} $
```

$\overpictarrow{v} \qquad \overpictarrow{AB}$

Keys to use LaTeX `picture` environment are described in section 4.3.6, page 30. The main keys are `picture command`→P. 30, `geometry`→P. 30 an `line thickness`→P. 30. Here is an example:

```
\NewOverArrowCommand[picture]{\overbandedarrow}{
  picture command={%
    \qbezier
    (0.0\overarrowlength,0)
    (0.5\overarrowlength,0)
    (0.9\overarrowlength,0.2\overarrowlength)
    \put(0.9\overarrowlength,0.2\overarrowlength)
    {\vector(2,1){0.2\overarrowlength}}
  },
  geometry={(\overarrowlength,0.4\overarrowlength)(0,0)},
  line thickness={\overarrowsmallerthickness},
  center arrow,
  space after arrow=0.4ex,
}
$ \overbandedarrow{v} \qquad \overbandedarrow{AB} $
```

$\overbandedarrow{v} \qquad \overbandedarrow{AB}$

4 User interface

4.1 Package options

The overarrows package accepts many options, given as a comma-separated list ⟨*options*⟩ at package loading: `\usepackage[`⟨*options*⟩`]{overarrows}`.

The option `esvect` is set by default. This can be overridden with `noesvect`.

4.1.1 esvect configuration

`esvect`

Loads the esvect package and redefines its vector commands `\vv`→P. 21 through the overarrows mechanism. Original esvect `\vv` macro is still available with `\esvectvv`→P. 21. The esvect font description is fixed to allow any font sizes.

The esvect package provides the symbol `\relbareda` - which is smaller and often more flexible than the classic one `\relbar` —. `\relbareda` fits with the standard *Computer Modern* math font, but can be unsuitable with other fonts.

The esvect package also provides the right arrow command `\fldr`. The shape of the arrow depends on the option passed to the esvect package: → (option `a`), → (option `b`), → (option `c`), → (option `d`), → (option `e`), → (option `f`), → (option `g`) or → (option `h`). Note that by default overarrows loads the esvect package with the option `f` (while esvect default is `d`). This can be changed with one of the eight options described bellow: `esvecta`, `esvectb`, `esvectc`, `esvectd`, `esvecte`, `esvectf`, `esvectg` and `esvecth`.

This option is set by default and can be unset with `noesvect`.

noesvect

Prevents the loading of the esvect package and the definition of the command `\vv` →P. 21.

esvecta

Loads the esvect package with the `a` option.
`\fldr` corresponds the to the symbol →. `\vv` command gives : $\vec{v}$ $\overrightarrow{AB}$ $\overrightarrow{\text{grad}}$.

esvectb

Loads the esvect package with the `b` option.
`\fldr` corresponds the to the symbol →. `\vv` command gives : $\vec{v}$ $\overrightarrow{AB}$ $\overrightarrow{\text{grad}}$.

esvectc

Loads the esvect package with the `c` option.
`\fldr` corresponds the to the symbol →. `\vv` command gives : $\vec{v}$ $\overrightarrow{AB}$ $\overrightarrow{\text{grad}}$.

esvectd

Loads the esvect package with the `d` option.
`\fldr` corresponds the to the symbol →. `\vv` command gives : $\vec{v}$ $\overrightarrow{AB}$ $\overrightarrow{\text{grad}}$.

esvecte

Loads the esvect package with the `e` option.
`\fldr` corresponds the to the symbol →. `\vv` command gives : $\vec{v}$ $\overrightarrow{AB}$ $\overrightarrow{\text{grad}}$.

esvectf

Loads the esvect package with the `f` option.
`\fldr` corresponds the to the symbol →. `\vv` command gives : $\vec{v}$ $\overrightarrow{AB}$ $\overrightarrow{\text{grad}}$.

esvectg

Loads the esvect package with the `g` option.
`\fldr` corresponds the to the symbol →. `\vv` command gives : $\vec{v}$ $\overrightarrow{AB}$ $\overrightarrow{\text{grad}}$.

esvecth

Loads the esvect package with the `h` option.
`\fldr` corresponds the to the symbol →. `\vv` command gives : $\vec{v}$ $\overrightarrow{AB}$ $\overrightarrow{\text{grad}}$.

4.1.2 Predefined commands

The overarrows package provides sixteen predefined commands, eight with the arrow over, and eight with the arrow under. By default, theses commands are not defined, and must be activated by the corresponding option. Beware that commands are created without checking if already defined by another package (`\overleftarrow`, `\overrightarrow`, `\overleftrightarrow`, `\underleftarrow`, `\underrightarrow` and `\underleftrightarrow` are, for example, part of the amsmath package).

Three options are also available to define set of commands.

Set of commands

allcommands

Defines all sixteen predefined commands.

overcommands

Defines all eight predefined commands with arrow over.

undercommands

Defines all eight predefined commands with arrow under.

Over arrows

overrightarrow

Defines the `\overrightarrow`$^{\rightarrow \text{P. } 21}$ command: $\overrightarrow{v}$, $\overrightarrow{AB}$, $\overrightarrow{\text{grad}}$.

overleftarrow

Defines the `\overleftarrow`$^{\rightarrow \text{P. } 22}$ command: $\overleftarrow{v}$, $\overleftarrow{AB}$, $\overleftarrow{\text{grad}}$.

overleftrightarrow

Defines the `\overleftrightarrow`$^{\rightarrow \text{P. } 22}$ command: $\overleftrightarrow{v}$, $\overleftrightarrow{AB}$, $\overleftrightarrow{\text{grad}}$.

overrightharpoonup

Defines the `\overrightharpoonup`$^{\rightarrow \text{P. } 22}$ command: $\overrightharpoonup{v}$, $\overrightharpoonup{AB}$, $\overrightharpoonup{\text{grad}}$.

overrightharpoondown

Defines the `\overrightharpoondown`$^{\rightarrow \text{P. } 22}$ command: $\overrightharpoondown{v}$, $\overrightharpoondown{AB}$, $\overrightharpoondown{\text{grad}}$.

overleftharpoonup

Defines the `\overleftharpoonup`$^{\rightarrow \text{P. } 22}$ command: $\overleftharpoonup{v}$, $\overleftharpoonup{AB}$, $\overleftharpoonup{\text{grad}}$.

overleftharpoondown

Defines the `\overleftharpoondown`$^{\rightarrow \text{P. } 22}$ command: $\overleftharpoondown{v}$, $\overleftharpoondown{AB}$, $\overleftharpoondown{\text{grad}}$.

overbar

Defines the `\overbar`$^{\rightarrow \text{P. } 22}$ command: $\overline{v}$, $\overline{AB}$, $\overline{\text{grad}}$.

Under arrows

underrightarrow

Defines the `\underrightarrow`$^{\rightarrow \text{P. } 22}$ command: $\underrightarrow{v}$, $\underrightarrow{AB}$, $\underrightarrow{\text{grad}}$.

underleftarrow

Defines the `\underleftarrow`$^{\rightarrow \text{P. } 22}$ command: $\underleftarrow{v}$, $\underleftarrow{AB}$, $\underleftarrow{\text{grad}}$.

underleftrightarrow

Defines the `\underleftrightarrow`$^{\rightarrow \text{P. } 22}$ command: $\underleftrightarrow{v}$, $\underleftrightarrow{AB}$, $\underleftrightarrow{\text{grad}}$.

underrightharpoonup

Defines the `\underrightharpoonup` command: $\underrightharpoonup{v}$, $\underrightharpoonup{AB}$, $\underrightharpoonup{\text{grad}}$.

underrightharpoondown

Defines the `\underrightharpoondown` command: $\underrightharpoondown{v}$, $\underrightharpoondown{AB}$, $\underrightharpoondown{\text{grad}}$.

underleftharpoonup

Defines the `\underleftharpoonup` command: $\underleftharpoonup{v}$, $\underleftharpoonup{AB}$, $\underleftharpoonup{\text{grad}}$.

underleftharpoondown

Defines the `\underleftharpoondown` command: $\underleftharpoondown{v}$, $\underleftharpoondown{AB}$, $\underleftharpoondown{\text{grad}}$.

underbar

Defines the `\underbar` command: $\underbar{v}$, $\underbar{AB}$, $\underbar{\text{grad}}$.

4.1.3 Other options

old-arrows

Loads the old-arrows package with its option `old`. This provides the symbols `\varleftarrow` ← and `\varrightarrow` →, used then by default for predefined command.

When the `old-arrows` option is set, the commands `\overrightarrow`, `\overleftarrow`, `\overleftrightarrow`, `\underrightarrow`, `\underleftarrow` and `\underleftrightarrow` give respectively : $\overrightarrow{AB}$, $\overleftarrow{AB}$, $\overleftrightarrow{AB}$, $\underrightarrow{AB}$, $\underleftarrow{AB}$ and $\underleftrightarrow{AB}$

tikz

Loads the package tikz with its library `arrows.meta`.

Note that TikZ arrows, drawn with the `tikz` method, are always available, even if this option is not set, provided the tikz package and its library are loaded independently.

pstricks

Loads the package pstricks-add.

Note that, as it, this will compile with LaTeX, LuaLaTeX and XeLaTeX, but not with pdfLaTeX (see the PSTricks documentation). PSTricks arrows, drawn with the `pstricks` method, are always available, even if this option is not set, provided the pstricks package is loaded independently.

pstarrows

Loads the pict2e package, with its option `pstarrows`. Vectors using LaTeX `picture` environment gives then $\overrightarrow{AB}$ instead of $\overrightarrow{AB}$.

Note that this affect all vectors drawn in LaTeX `picture` environments, and that this setting can be changed on the fly with the commands `\pstarrows` and `\ltxarrows` from the pict2e package.

subscripts

Sets the default value of the key `detect subscripts` → P. 25 to `true`.

This option also impacts the command `\vv` → P. 21 and all predefined commands, so that they automatically use their starred variant when a subscript follows.

subother

New: v1.1 2023/02/15

Sets to 12 (*other* catcode category) the catcode of the "`_`" symbol used for subscript detection, when this is enabled by the key `detect subscripts` → P. 25 (see the section 5.1.2, page 32).

subactive

New: v1.1 2023/02/15

Sets to 13 (*active* catcode category) the catcode of the "`_`" symbol used for subscript detection, when this is enabled by the key `detect subscripts` → P. 25 (see the section 5.1.2, page 32).

debug

Writes the meaning of defined commands in LaTeX log.

4.2 Commands

4.2.1 Macro for commands creation

`\NewOverArrowCommand[`⟨*method*⟩`]{`⟨*command*⟩`}{`⟨*keys*⟩`}`
`\RenewOverArrowCommand[`⟨*method*⟩`]{`⟨*command*⟩`}{`⟨*keys*⟩`}`
`\ProvideOverArrowCommand[`⟨*method*⟩`]{`⟨*command*⟩`}{`⟨*keys*⟩`}`
`\DeclareOverArrowCommand[`⟨*method*⟩`]{`⟨*command*⟩`}{`⟨*keys*⟩`}`

Creates the command ⟨*command*⟩ and its starred variant ⟨*command*⟩`*`. The starred variant ⟨*command*⟩`*` removes the extra end space generated by the arrow, which is suitable, as example, when a subscript follows.

Updated: v1.2 2024/07/11

⟨*command*⟩ can be given with or without backslash (prior to the version 1.2, only the name, without backslash, was accepted).

Updated: v1.4 2025/04/30

Since the version 1.4, the ⟨*command*⟩ is robust (for example, it can be used safety inside `\caption`).

\NewOverArrowCommand raises an error if ⟨*command*⟩ is already defined.

\RenewOverArrowCommand raises an error if ⟨*command*⟩ is undefined.

\ProvideOverArrowCommand sets ⟨*command*⟩ if the command is undefined and does nothing if it is already defined, without raising any error.

\DeclareOverArrowCommand sets ⟨*command*⟩, whether the command is already defined or not, without raising any error.

The ⟨*method*⟩ used to draw the arrow must be:

`symb` to draw the arrow by symbols assemblage (default);

`tikz` to draw the arrow with PGF/TikZ;

New: v1.2 2024/07/11

`pstricks` to draw the arrow with PSTricks;

`picture` to draw the arrow with the LaTeX `picture` environment.

With no ⟨*method*⟩ argument, the `symb` method is chosen.

⟨*keys*⟩ is a comma-separated list of keys-values. Available keys depends of the ⟨*method*⟩ chosen and are described in section 4.3, page 23.

```
\NewOverArrowCommand[tikz]{\myoverarrow}{arrows={Bar-Bar}, center arrow}
$ \myoverarrow{v} \qquad \myoverarrow{ABCD} $
```

$$\overbrace{v} \qquad \overbrace{ABCD}$$

`\TestOverArrow[`⟨*pattern*⟩`]{`⟨*command*⟩`}`
`\TestOverArrow*[`⟨*pattern*⟩`]{`⟨*command*⟩`}`

Displays the result of the command ⟨*command*⟩ for patterns of various lengths and for the four math styles. A custom ⟨*pattern*⟩ can be added to the predefined ones.

The starred variant `\TestOverArrow*` displays a full report, including kerning tests of the commands ⟨*command*⟩ and ⟨*command*⟩`*`.

Updated: v1.2 2024/07/11

⟨*command*⟩ can be given with or without backslash (prior to the version 1.2, only the name, without backslash, was accepted).

```
\TestOverArrow*[my~pattern]{vv}
```

Test of \vv and \vv* macros

\vv for different math styles

\displaystyle	\textstyle	\scriptstyle	\scriptscriptstyle
$\overrightarrow{v}$	$\overrightarrow{v}$	$\overrightarrow{v}$	$\overrightarrow{v}$
$\overrightarrow{AB}$	$\overrightarrow{AB}$	$\overrightarrow{AB}$	$\overrightarrow{AB}$
$\overrightarrow{\mathrm{grad}}$	$\overrightarrow{\mathrm{grad}}$	$\overrightarrow{\mathrm{grad}}$	$\overrightarrow{\mathrm{grad}}$
$\overrightarrow{my\ long\ vector}$	$\overrightarrow{my\ long\ vector}$	$\overrightarrow{my\ long\ vector}$	$\overrightarrow{my\ long\ vector}$
$\overrightarrow{my\ pattern}$	$\overrightarrow{my\ pattern}$	$\overrightarrow{my\ pattern}$	$\overrightarrow{my\ pattern}$

\vv kerning

$\overrightarrow{t}_{\overrightarrow{u}\,\overrightarrow{v}} \qquad \overrightarrow{\imath}_0 \qquad \overrightarrow{v} = \overrightarrow{v}_x + \overrightarrow{v}_y + \overrightarrow{v}_z = v_x\overrightarrow{\imath} + v_y\overrightarrow{\jmath} + v_z\overrightarrow{k}$

\vv* kerning

$\overrightarrow{t}_{\overrightarrow{u}\,\overrightarrow{v}} \qquad \overrightarrow{\imath}_0 \qquad \overrightarrow{v} = \overrightarrow{v}_x + \overrightarrow{v}_y + \overrightarrow{v}_z = v_x\overrightarrow{\imath} + v_y\overrightarrow{\jmath} + v_z\overrightarrow{k}$

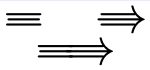
4.2.2 Useful macros for symbols assemblage

Math symbols assemblage is the default method used to draw arrows. The following macros are designed to help combine and format math symbols.

\xjoinrel[⟨number⟩]

Removes an horizontal space of ⟨*number*⟩ math units (`3.5 mu` by default). Must be used in math mode. Useful to assemble math symbols and create new ones.

```
\newcommand*{\triplebar}{\Relbar\xjoinrel[14]\relbar}
\newcommand*{\triplebararrow}{\Relbar\xjoinrel[15]\rightarrow}
\scalebox{2}{$ \triplebar \quad \triplebararrow $} \par
\scalebox{2}{$ \triplebar\xjoinrel\triplebararrow $}
```

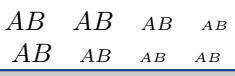


\smallermathstyle

Applies the next math style, smaller than the current. That is:

- uses `\scriptstyle` if the current math style is `\displaystyle` or `\textstyle`;
- uses `\scriptscriptstyle` if the current math style is `\scriptstyle`;
- does nothing if the current math style is `\scriptscriptstyle`.

```
$ \displaystyle AB \quad \textstyle AB
 \quad \scriptstyle AB \quad \scriptscriptstyle AB $\par
$ \displaystyle AB \quad \smallermathstyle AB
 \quad \smallermathstyle AB \quad \smallermathstyle AB $
```

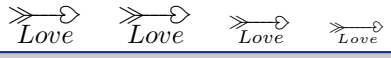


\arrowmathstyle

New: v1.5 2025/10/19

Expands to the current math style. This could be useful inside box macros, such as, for example, `\raisebox`, `\rotatebox` or `\scalebox` (these last two are defined by the package graphicx). `\arrowmathstyle` is only available inside commands created with `\NewOverArrowCommand` → P. 17 and variants.

```
% ^^A \rotatebox is defined with \usepackage{graphicx}
\NewOverArrowCommand{\heartarrow}{%
  start={\raisebox{\dimeval{-0.5\height+0.5\depth}}{$\arrowmathstyle\gg$}},
  end={\rotatebox[origin=cB]{+90}{$\arrowmathstyle\heartsuit$}},
  middle={\baselinebar},
  trim start=7,
  trim end=2.5,
  trim middle=4,
  shift leftright=-6,
}
$ \displaystyle \heartarrow{Love}
\qquad \textstyle \heartarrow{Love}
\qquad \scriptstyle \heartarrow{Love}
\qquad \scriptscriptstyle \heartarrow{Love} $
```



\baselinebar

New: v1.5 2025/10/19

Draws a zero-height horizontal bar, aligned with the baseline (analogous to `\relbar` defined by LaTeX, and redefined by the amsmath package, but centered on the baseline). `\baselinebar` should work with most fonts. It is used in the definition of `\overbar` → P. 22 and `\underbar` → P. 23.

For example, the `\overbar` → P. 22 command is defined with:

```
\DeclareOverArrowCommand{\overbar}{%
  amsmath, middle={\baselinebar},
  start={\baselinebar}, end={\baselinebar},
  space before arrow={0.5\overarrowthickness},
  space after arrow={3.5\overarrowthickness}
}
```

4.2.3 Useful lengths for TikZ, PSTricks or `picture` environment

Arrows drawn with graphic languages, like PGF/TikZ, PSTricks or the LaTeX `picture` environment, are not extensible. The three lengths `\overarrowlength`, `\overarrowthickness` and `\overarrowsmallerthickness` are computed at each utilisation of a command set with the `tikz`, `pstricks` or `picture` method, so they can be used in drawing commands.

```
\NewOverArrowCommand[tikz]{\overparabola}{%
  path options={x=\overarrowlength, line width=\overarrowsmallerthickness},
  path={(0,0) parabola[parabola height=0.2\overarrowlength] (1,0)},
  arrows={-}, center arrow, min length=30,
}
$\displaystyle \overparabola{v} \qquad \overparabola{ABCD} $ \par
$\scriptstyle \overparabola{v} \qquad \overparabola{ABCD} $ \par
```



`\overarrowlength`

Is set to the width of the arrow command content, or, if larger, to the minimal arrow length set through the key `min length` → P. 23.

`\overarrowthickness`

Is set to the default rule thickness of the current math style. That is:

- `\fontdimen 8 \textfont 3` in `\displaystyle` or `\textstyle`;
- `\fontdimen 8 \scriptfont 3` in `\scriptstyle`;
- `\fontdimen 8 \scriptscriptfont 3` in `\scriptscriptstyle`.

Updated: v1.2 2024/07/11

Theses settings are adapted when the package unicode-math is loaded (using `\Umathoverbarrule` with LuaLaTeX or `\fontdimen 54`, family 2 with XeLaTeX — see the manual of unicode-math).

Updated: v1.5 2025/10/19

Since the version 1.5, the length `\overarrowthickness` is always available, whatever drawing method is used for the arrow (including arrows drawn with symbols assemblage).

`\overarrowsmallerthickness`

Is set to the default rule thickness of the next smaller math style. That is:

- `\fontdimen 8 \scriptfont 3` in `\displaystyle` or `\textstyle`;
- `\fontdimen 8 \scriptscriptfont 3` in `\scriptstyle` or `\scriptscriptstyle`.

Updated: v1.2 2024/07/11

Theses settings are adapted when the package unicode-math is loaded (using `\Umathoverbarrule` with LuaLaTeX or `\fontdimen 54`, family 2 with XeLaTeX — see the manual of unicode-math).

Updated: v1.5 2025/10/19

Since the version 1.5, the length `\overarrowsmallerthickness` is always available, whatever drawing method is used for the arrow (including arrows drawn with symbols assemblage).

4.2.4 Vectors macros

The macro `\vv`, dedicated to vectors, is automatically defined when the option `esvect`→P. 13 is set (which is the default). It is a clone of the `\vv` command provided by the esvect package, but its starred variant has a correct kerning when followed by a subscript.

`\vv{`⟨*content*⟩`}`
`\vv*{`⟨*content*⟩`}`

Draws a vector arrow upon math ⟨*content*⟩. The shape of the arrow depends on the corresponding options described in section 4.1.1, page 13 : `esvecta`→P. 14, `esvectb`→P. 14, `esvectc`→P. 14, `esvectd`→P. 14, `esvecte`→P. 14, `esvectf`→P. 14, `esvectg`→P. 14, `esvecth`→P. 14.

The starred variant `\vv*` suppresses the end space created by the arrow.

```
$ \vv{\imath}_{0}  \quad \vv{e}_r \quad \vv{L}_\Delta $\par
$ \vv*{\imath}_{0}  \quad \vv*{e}_r \quad \vv*{L}_\Delta $
```

$$\vec{\imath}_0 \quad \vec{e}_r \quad \vec{L}_\Delta$$
$$\vec{\imath}_0 \quad \vec{e}_r \quad \vec{L}_\Delta$$

`\esvectvv`

Is simply the backup of the original esvect `\vv` command.

```
$ \esvectvv{\imath}_{0}  \quad \esvectvv{e}_{r} \quad \esvectvv{L}_\Delta $\par
$ \esvectvv*{\imath}{0}  \quad \esvectvv*{e}{r} \quad \esvectvv*{L}{\Delta} $
```

$$\vec{\imath}_0 \quad \vec{e}_r \quad \vec{L}_\Delta$$
$$\vec{\imath}_0 \quad \vec{e}_r \quad \vec{L}_\Delta$$

4.2.5 Predefined commands

Predefined commands are defined if the corresponding option is set (see section 4.1.2, page 14). The commands `\overrightarrow`, `\overleftarrow`, `\overleftrightarrow`, `\underrightarrow`, `\underleftarrow` and `\underleftrightarrow` are affected by the option `old-arrows`→P. 16.

Over arrows

`\overrightarrow`

$$\overrightarrow{v} \qquad \overrightarrow{AB} \qquad \overrightarrow{\text{grad}}$$

The shape of the arrow is smaller if the option `old-arrows`→P. 16 is set.

`\overleftarrow`

$$\overleftarrow{v} \qquad \overleftarrow{AB} \qquad \overleftarrow{\text{grad}}$$

The shape of the arrow is smaller if the option `old-arrows` →P. 16 is set.

`\overleftrightarrow`

$$\overleftrightarrow{v} \qquad \overleftrightarrow{AB} \qquad \overleftrightarrow{\text{grad}}$$

The shape of the arrows is smaller if the option `old-arrows` →P. 16 is set.

`\overrightharpoonup`

$$\overrightharpoonup{v} \qquad \overrightharpoonup{AB} \qquad \overrightharpoonup{\text{grad}}$$

`\overrightharpoondown`

$$\overrightharpoondown{v} \qquad \overrightharpoondown{AB} \qquad \overrightharpoondown{\text{grad}}$$

`\overleftharpoonup`

$$\overleftharpoonup{v} \qquad \overleftharpoonup{AB} \qquad \overleftharpoonup{\text{grad}}$$

`\overleftharpoondown`

$$\overleftharpoondown{v} \qquad \overleftharpoondown{AB} \qquad \overleftharpoondown{\text{grad}}$$

`\overbar`

$$\overline{v} \qquad \overline{AB} \qquad \overline{\text{grad}}$$

Updated: v1.5 2025/10/19

Since version 1.5, the `\overbar` command car be properly nested.

$$\overline{\overline{AB}} \qquad x_{\overline{\overline{AB}}}$$

Under arrows

`\underrightarrow`

$$\underrightarrow{v} \qquad \underrightarrow{AB} \qquad \underrightarrow{\text{grad}}$$

The shape of the arrow is smaller if the option `old-arrows` →P. 16 is set.

`\underleftarrow`

$$\underleftarrow{v} \qquad \underleftarrow{AB} \qquad \underleftarrow{\text{grad}}$$

The shape of the arrow is smaller if the option `old-arrows` →P. 16 is set.

`\underleftrightarrow`

$$\underleftrightarrow{v} \qquad \underleftrightarrow{AB} \qquad \underleftrightarrow{\text{grad}}$$

The shape of the arrows is smaller if the option `old-arrows` →P. 16 is set.

`\underrightharpoonup`

$\underrightharpoonup{v}$ $\underrightharpoonup{AB}$ $\underrightharpoonup{\text{grad}}$

`\underrightharpoondown`

$\underrightharpoondown{v}$ $\underrightharpoondown{AB}$ $\underrightharpoondown{\text{grad}}$

`\underleftharpoonup`

$\underleftharpoonup{v}$ $\underleftharpoonup{AB}$ $\underleftharpoonup{\text{grad}}$

`\underleftharpoondown`

$\underleftharpoondown{v}$ $\underleftharpoondown{AB}$ $\underleftharpoondown{\text{grad}}$

`\underbar`

$\underbar{v}$ $\underbar{AB}$ $\underbar{\text{grad}}$

Updated: v1.5 2025/10/19

Since version 1.5, the `\underbar` command car be properly nested.

$\underbar{\underbar{AB}}$ $x_{\underbar{\underbar{AB}}}$

4.3 Keys

The customisation of arrows is done at command creation through a key-value interface provided by the pgfkeys package (with `/overarrows/` as key path).

4.3.1 Arrow position and length settings

These keys are available whatever the method chosen at command creation (see section 4.2.1, page 17 for the documentation of commands creation).

Length

`min length=`{⟨*number*⟩} (no default, see below for the initial value)

Sets the minimal arrow length to ⟨*number*⟩ math units. The arrow length is set from content width, or, if larger, to this value.

The initial value of `min length` depends on the ⟨*method*⟩ chosen at command creation (see section 4.2.1, page 17 for the documentation of commands creation):

- ⟨*number*⟩ = 0 for the `symb` method (method by default);
- ⟨*number*⟩ = 12 for the `tikz` method;
- ⟨*number*⟩ = 12 for the `pstricks` method;
- ⟨*number*⟩ = 18 for the `picture` method.

```
\NewOverArrowCommand{\overlongarrow}{min length=50}
$ \overlongarrow{v} \qquad \overlongarrow{ABCDEF} $
```

$\overlongarrow{v}$ $\overlongarrow{ABCDEF}$

Placement

`arrow under` (default `autoconfig`, initially unset)
`arrow under=autoconfig|noconfig`

Places the arrow under, instead of over.

`arrow under` or `arrow under=autoconfig` also configures suitably the key `detect subscripts` →P. 25 to `false` and the key `before arrow` →P. 25 to get an additional space over the arrow.

`arrow under=noconfig` does not do any additional configuration.

```
\NewOverArrowCommand{\underhooks}{%
  start={\lhook}, end={\rhook},  trim=1,
  arrow under, shift leftright=-4,
}
$ \underhooks{v} \qquad \underhooks{AB} $
```

$\underset{\smile}{v} \qquad \underset{\frown}{AB}$

Horizontal shifts

`shift left={`⟨*number*⟩`}` (no default, initially 2)

Shifts the left side of the arrow by ⟨*number*⟩ math units (positive number means a shift to the right).

`shift right={`⟨*number*⟩`}` (no default, see below for the initial value)

Shifts the right side of the arrow by ⟨*number*⟩ math units (positive number means a shift to the left).

The initial value of `shift right` depends on the ⟨*method*⟩ chosen at command creation (see section 4.2.1, page 17 for the documentation of commands creation):

- ⟨*number*⟩ `= 0` for the `symb` method (method by default);
- ⟨*number*⟩ `= -2` for the `tikz`, `pstricks` and `picture` methods.

```
\NewOverArrowCommand{\lookback}{%
  start={\leftarrow}, end={\rightharpoondown},
  shift left=-50, shift right=-10,
}
$ \lookback{\text{look back}} $
```

$\overset{\longleftarrow\!\!\!

`right arrow=`{⟨*number*⟩} (default 2)

Sets `shift right` to zero and `shift left` to ⟨*number*⟩.

Vertical adjunct

`before arrow=`{⟨*vertical material*⟩} (initially empty)
`after arrow=`{⟨*vertical material*⟩} (initially empty)

Adds the ⟨*vertical material*⟩ before or after the arrow.

Over and under arrow commands are typeset through the TEX `\ialign` command, which aligns contents, like a tabular. The ⟨*vertical material*⟩ is inserted *between* the rows, with TEX `\noalign` command.

These keys are essentially used to add some extra space between the arrow and the content of the command. They can be set in a handier way with the keys `space before arrow` and `space after arrow`.

`space before arrow=`{⟨*length*⟩} (no default)

Adds a space of ⟨*length*⟩ before the arrow. This sets the keys `before arrow`.

`space after arrow=`{⟨*length*⟩} (no default)

Adds a space of ⟨*length*⟩ after the arrow. This sets the keys `after arrow`.

Updated: v1.5 2025/10/19

Since the version 1.5, the length `\overarrowthickness` → P. 20 can be used to set `space before arrow` or `space after arrow`. The main interest is that `\overarrowthickness` → P. 20 scale with the math style and the main font.

```
\NewOverArrowCommand{\overharpoonsdown}{%
  start=\leftharpoondown, end=\rightharpoondown, center arrow,
  space before arrow=-\overarrowthickness,
  space after arrow=3\overarrowthickness,
}
$ \dot{\overharpoonsdown{v}} \quad \ddot{\overharpoonsdown{AB}} \qquad
\scriptstyle \dot{\overharpoonsdown{v}} \quad \ddot{\overharpoonsdown{AB}} $
```

$\overset{\dot{}}{\overset{\rightharpoondown}{v}}$ $\overset{\ddot{}}{\overset{\rightharpoondown}{AB}}$ $\overset{\dot{}}{\overset{\rightharpoondown}{v}}$ $\overset{\ddot{}}{\overset{\rightharpoondown}{AB}}$

4.3.2 Subscripts detection setting

This key is available whatever the method chosen at command creation (see section 4.2.1, page 17 for the documentation of commands creation).

`detect subscripts=true|false` (default `true`, see below for the initial value)

Removes automatically the extra end space created by the arrow, if a subscript immediately follows the command.

By default, the initial value of `detect subscripts` is `false`. When the option `subscripts` → P. 17 is set, the initial value of `detect subscripts` is `true`.

Note that the detection may fail when the standard subscript command is changed or altered (see the section 5.1.2, page 32).

```
\NewOverArrowCommand{\autosub}{detect subscripts}
$ \imath_0 \qquad \autosub{\imath}_0 \qquad
 {\autosub{\imath}}_0 \qquad {\autosub*{\imath}}_0 $
```

$\imath_0$ $\overset{\rightharpoondown}{\imath_0}$ $\overset{\rightharpoondown}{\imath}_0$ $\overset{\rightharpoondown}{\imath_0}$

4.3.3 Symbols assemblage settings

The following keys are available for arrows drawn with the default `symb` method (see section 4.2.1, page 17 for the documentation of commands creation).

`start={`⟨*command*⟩`}` (no default, initially `\relbar`)
`middle={`⟨*command*⟩`}` (no default, initially set by `middle config=auto`)
`end={`⟨*command*⟩`}` (no default, see below for the initial value)

Sets the ⟨*command*⟩ used to draw the start (left), middle (center) or end (right) part of the arrow. The `middle` one is repeated, if necessary, to extend the arrow. It is set, initially by `middle config=auto`. By default, the `end` symbols is initially `\rightarrow` →. When the option `old-arrows` → P. 16 is set, the initial value of `end` is `\varrightarrow` →.

`start` and `end` symbols are typeset in the same group. `middle` is typeset alone. This means that, if a command, like `\smallermathstyle` → P. 19, is used to alter the symbols, it should be applied both to `start` and `middle` (but not to `end`).

```
\NewOverArrowCommand{\smalleroverrightarrow}{%
  start={\smallermathstyle\relbar},
  middle={\smallermathstyle\relbareda},
  end={\rightarrow},
  space after arrow={0.2ex},
}
$ \smalleroverrightarrow{v} \qquad \smalleroverrightarrow{AB} $
```

$\vec{v} \qquad \overrightarrow{AB}$

`trim start={`⟨*number*⟩`}` (no default, initially 7)

Trims ⟨*number*⟩ math units from the right side of the `start` symbol.

`trim middle={`⟨*number*⟩`}` (no default, initially set by `middle config=auto`)

Trims ⟨*number*⟩ math units from both left and right sides of the `middle` symbol.

`trim end={`⟨*number*⟩`}` (no default, initially 7)

Trims ⟨*number*⟩ math units from the left side of the `end` symbol.

`trim={`⟨*number*⟩`}` (no default)

Sets `trim start`, `trim middle` and `trim end` to the same ⟨*number*⟩ value.

`no trimming`

Clears `trim start`, `trim middle` and `trim end`.

`middle config=auto|relbar|relbareda|harrowextender` (no default)

Sets a suitable configuration for the keys `middle` and `trim middle`:

For `middle config = relbar`, `middle` is set to `\relbar` − and `trim middle` to 2.5.

For `middle config = relbareda`, `middle` is set to `\relbareda` - and `trim middle` to 1.

New: v1.2 2024/07/11

For `middle config = harrowextender`, `middle` is set to `\harrowextender` and `trim middle` to 0.

Updated: v1.2 2024/07/11

For `middle config = auto`, if `\harrowextender` is provided by the math font[10], `middle` is set with `middle config = harrowextender`. If `\harrowextender` isn't availlable, `middle` is set with `middle config = relbareda` if the option `esvect`→ P. 13 is set (which is the default) and `middle config = relabar` if not.

`amsmath` (default `mimic`)
`amsmath=mimic|strict`

Loads a configuration coherent with amsmath `\overrightarrow` command.

`amsmath` or `amsmath=mimic` sets the corresponding keys suitably:

```
start={\relbar}      middle={\relbar}    end={\rightarrow}
trim start=7         trim middle=2       trim end=7
shift leftright=0    after arrow={}      before arrow={}
```

`amsmath=strict` makes, in addition, the command uses the internal macros of amsmath `\overrightarrow` (`no trimming`, `fill macro={\arrowfill@}`, `stack macro={\overarrow@}`). Note that many configuration keys becomes ineffective.

`esvect` (default `mimic`)
`esvect=mimic|strict`

Loads a configuration coherent with amsmath `\vv` command.

`esvect` or `esvect=mimic` sets the corresponding keys suitably:

```
start={\relbaredd}        middle={\relbareda}      end={\fldr}
trim start=1.5            trim middle=0            trim end=1.5
space before arrow=-.7pt  space after arrow=-.3pt  right arrow=2
```

`esvect=strict` makes, in addition, the command uses the internal macros of esvect `\vv` (`no trimming`, `fill macro={\traitfill@}`, `stack macro={\overvect@}`). Note that many configuration keys becomes ineffective.

4.3.4 TikZ settings

If, at command creation (see section 4.2.1, page 17 for the documentation of commands creation), the `tikz` method is chosen, then the arrow is drawn by the command:

```
\tikz[<tikz options>]{<tikz command>}
```

where `tikz options`→ P. 28 and `tikz command`→ P. 28 are two keys described below. When `tikz command` is let unset, the drawing command turns into:

```
\tikz[<tikz options>]{\draw[<path options>] <path>;}
```

[10] See the documentation of the package unicode-math.

The best way to customise `tikz` arrows is then to set the keys `tikz options`→P. 28, `path options`→P. 28 and `path`→P. 28, preferably through the handy alternatives: `add tikz options`→P. 28, `add path options`→P. 28, `arrows`→P. 28, `line thickness`→P. 28 or `thinner`→P. 28.

```
\NewOverArrowCommand[tikz]{\overdotteddoublearrow}{%
  add tikz options={blue}, add path options={densely dotted},
  arrows={->[scale=0.5]>[scale=0.5]}, thinner,
  min length=20, space after arrow={0.3ex},
}
$ \overdotteddoublearrow{v} \qquad \overdotteddoublearrow{AB} $
```



The following keys are available when the `tikz` method is chosen.

`tikz options={`⟨*TikZ options*⟩`}`
(no default, initially `x=\overarrowlength, line width=\overarrowthickness`)

Sets TikZ options to ⟨*TikZ options*⟩.

`path options={`⟨*path options*⟩`}`
(no default, initially `arrows=-Classical TikZ Rightarrow, cap=round`)

Sets TikZ path options to ⟨*path options*⟩.

`path={`⟨*path specification*⟩`}` (no default, initially `(0,0)--(1,0)`)

Sets TikZ path specification to ⟨*path*⟩ (the ending semicolon is automatically appended).

`add tikz options={`⟨*TikZ options*⟩`}` (no default)

Appends the options ⟨*TikZ options*⟩ to the key `tikz options`.

`add path options={`⟨*path options*⟩`}` (no default)

Appends the options ⟨*path options*⟩ to the key `path options`.

`arrows={`⟨*arrow specification*⟩`}` (no default)

Appends the option `arrows={`⟨*arrow specification*⟩`}` to the key `path options`.

`line thickness={`⟨*length*⟩`}` (no default)

Appends the option `line width={`⟨*length*⟩`}` to the key `path options`.

`thinner`

Sets the keys `line thickness` with `\overarrowsmallerthickness`.

`tikz command={`⟨*TikZ command*⟩`}` (initially unset)

Sets the ⟨*TikZ command*⟩ used to draw the arrow. If left unset, the value `\draw[path options] path;` is used.

4.3.5 PSTricks settings

New: v1.2 2024/07/11

If, at command creation (see section 4.2.1, page 17 for the documentation of commands creation), the `pstricks` method is chosen, then the arrow is drawn by:

```
\begin{pspicture}<geometry>%
  \psset{linewidth=<line thickness>}%
  \psset{<psset>}%
  <pstricks command>%
\end{pspicture}%
```

where `geometry`, `line thickness` `psset` and `pstricks command` are four keys described below.

```
\NewOverArrowCommand[pstricks]{\overloopandarrow}{
  pstricks command={%
    \pscurve{->}(0,0)
    (0.6\overarrowlength,0.05\overarrowlength)
    (0.5\overarrowlength,0.1\overarrowlength)
    (0.4\overarrowlength,0.05\overarrowlength)
    (\overarrowlength,0)
  },
  geometry={(0,0)(\overarrowlength,0.2\overarrowlength)},
  space after arrow=2pt, min length=20,
  geometry={(0,0)(\overarrowlength,0.2\overarrowlength)},
}
$ \overloopandarrow{v} \qquad \overloopandarrow{AB} $
```

$\overset{\circ}{\overrightarrow{v}} \qquad \overset{\circ}{\overrightarrow{AB}}$

The following keys are available when the `pstricks` method is chosen.

`pstricks command={`⟨*pstricks command*⟩**`}`**
(no default, initially `\psline{->}(0,0)(\overarrowlength,0)`)

Sets the `pspicture` command to ⟨*pstricks command*⟩.

`arrow={`⟨*arrow*⟩**`}`** (no default, initially `->`)

Sets `pstricks command` with `\psline{`⟨*arrow*⟩`}(0,0)(\overarrowlength,0)`.

`psset={`⟨*pstricks setting*⟩**`}`** (no default, initially empty)

Sets ⟨*pstricks setting*⟩ with `\psset`.

`geometry={`⟨*pstricks geometry specification*⟩**`}`**
(no default, initially `(0,-0.5ex)(\overarrowlength,1ex)`)

Sets the `pspicture` geometry to ⟨*pstricks geometry specification*⟩.

`line thickness={`⟨*length*⟩**`}`** (no default)

Sets the line thickness to ⟨*length*⟩.

`thinner`

Sets the keys `line thickness` with `\overarrowsmallerthickness`.

4.3.6 Picture environment settings

If, at command creation (see section 4.2.1, page 17 for the documentation of commands creation), the `picture` method is chosen, then the arrow is drawn by:

```
\begin{picture}<geometry>%
  \linethickness{<line thickness>}%
  <picture command>%
\end{picture}%
```

where `geometry`, `line thickness` and `picture command` are three keys described below.

```
% ^^A \arc and \roundcap commands are from the pict2e package
% ^^A this example needs \usepackage{pict2e} in the preamble
\NewOverArrowCommand[picture]{\overarc}{%
  picture command={%
    \roundcap
    \put(0.5\overarrowlength,0){\arc[180,0]{0.6\overarrowlength}}
  },
  geometry={%
    (1.2\overarrowlength,0.5\overarrowlength)(-0.1\overarrowlength,0.2ex)
  },
  thinner, center arrow,
}
$ \overarc{v} \qquad \overarc{AB} $
```

$\overarc{v} \qquad \overarc{AB}$

The following keys are available when the `picture` method is chosen.

`picture command={`⟨*picture command*⟩**`}`**
(no default, initially `\put(0,0){\vector(1,0){\overarrowlength}}`)

Sets picture command to ⟨*picture command*⟩.

`geometry={`⟨*picture geometry specification*⟩**`}`**
(no default, initially `(\overarrowlength,1ex)(0,-0.5ex)`)

Sets picture geometry to ⟨*picture geometry specification*⟩.

`line thickness={`⟨*length*⟩**`}`** (no default)

Sets the picture line thickness to ⟨*length*⟩.

`thinner` (no default)

Sets the keys `line thickness` with `\overarrowsmallerthickness`.

4.4 Advanced commands and keys

The following commands and keys are used in the implementation of the overarrows package. They can also be employed for an advanced configuration of the commands created, although unnecessary in the vast majority of cases.

4.4.1 Advanced commands

`\SetOverArrowsSubscriptCommand{`⟨*command*⟩`}`

New: v1.1 2023/02/15

Sets to ⟨*command*⟩ the command used for subscript detection, when this is enabled by the key `detect subscripts`→P. 25 (see the section 5.1.2, page 32).

`\SetOverArrowsMethod[`⟨*stack mechanism*⟩`]{`⟨*name*⟩`}[`⟨*pre code*⟩`]{`⟨*keys def*⟩`}`
`\SetOverArrowsMethod*{`⟨*name*⟩`}[`⟨*pre code*⟩`]{`⟨*keys def*⟩`}`

Defines the method ⟨*name*⟩, to be used with `\NewOverArrowCommand`→P. 17, with `\RenewOverArrowCommand`→P. 17, with `\ProvideOverArrowCommand`→P. 17 or with `\DeclareOverArrowCommand`→P. 17. When the ⟨*name*⟩ method is chosen, corresponding keys are defined by ⟨*keys def*⟩. This must set, in particular, the keys `no stack macro hook` and `no arrow macro hook`→P. 32. Optional code ⟨*pre code*⟩ is evaluated before the keys definition.

The unstarred variant automatically defines the key `no stack macro hook`, according to the value of the optional ⟨*stack mechanism*⟩. This one must be:

`fill` if `arrow macro` creates extensible arrows (typically with `\cleaders`). In this case, the arrow macro (defined by `no arrow macro hook`→P. 32) is called with the math style, passed as argument (it can be, for example, the macro `\rightarrowfill@` used by amsmath `\overrightarrow`). `fill` is the mechanism used by the `symb` method.

`lens` if `arrow macro` creates fixed-length arrows, and needs the computation of lengths `\overarrowlength`→P. 20, `\overarrowthickness`→P. 20 and `\overarrowsmallerthickness`→P. 20. In this case, the arrow macro (defined by `no arrow macro hook`→P. 32) is called without argument. `lens` is the mechanism used by the `tikz` and `picture` methods.

Without optional ⟨*stack mechanism*⟩, `fill` is used. The starred variant does not set the key `no stack macro hook`.

4.4.2 Advanced keys

`stack macro=`{⟨*stack definition*⟩} (no default, initially unset)

Defines the stack macro to be ⟨*stack definition*⟩. Stack macro is a command which takes three arguments: the arrow macro set by `arrow macro`, the math style, and the command content (under or over the arrow). ⟨*stack definition*⟩ can be, for example, the macro `\overarrow@` used by amsmath `\overrightarrow`.

`arrow macro=`{⟨*arrow definition*⟩} (no default, initially unset)

Defines the arrow macro (used in the stack macro) by to be ⟨*arrow definition*⟩.

`no stack macro hook=`{⟨*code*⟩} (no default)

Sets the ⟨*code*⟩ executed if `stack macro` is left unset, after user evaluation of ⟨*keys*⟩ in `\NewOverArrowCommand`→P. 17, `\RenewOverArrowCommand`→P. 17, `\ProvideOverArrowCommand`→P. 17 or `\DeclareOverArrowCommand`→P. 17.

⟨*code*⟩ must configure `stack macro` accordingly to the user keys setting.

`no arrow macro hook={`⟨*code*⟩`}` (no default)

Sets the ⟨*code*⟩ executed if `arrow macro` is left unset, after user evaluation of ⟨*keys*⟩ in `\NewOverArrowCommand`→P. 17, `\RenewOverArrowCommand`→P. 17, `\ProvideOverArrowCommand`→P. 17 or `\DeclareOverArrowCommand`→P. 17.

⟨*code*⟩ must configure `arrow macro`→P. 31 accordingly to the user keys setting.

`fill macro={`⟨*definition*⟩`}` (no default, initially unset)

Defines the fill macro to be ⟨*definition*⟩. The fill macro is used by arrows created with the `symb` method, to set `arrow macro`→P. 31 in `no arrow macro hook`. It is called with fours arguments: start, middle and end symbols used to draw the arrow, and the math style. ⟨*definition*⟩ can be, for example, the macro `\arrowfill@` used by amsmath `\overrightarrow`.

5 Complements

5.1 Know issues

5.1.1 Math font change

If the math font differs from the default *Computer Modern*, arrow drawn with the `symb` method may have a central part of the arrow with inappropriate position or line width. This is because the default symbol used for the arrow line is `\relbareda` - from the esvect package. This can be fixed with the `noesvect`→P. 14 option.

Depending of the math font, predefined commands may be faulty. For example, at the time of writing, hooks vertical position is incorrect with *Asana Math* or `\harrowextender` is badly positioned with *Stix two Math* (for the smallest math styles), *Libertinus Math* and *GFSNeohellenicMath*.

5.1.2 Detection of non standard subscripts

The subscript detection enabled by the key `detect subscripts`→P. 25 is based on the LaTeX macro `\@ifnextchar`. The detection may fail if the standard subscript command is modified of altered. This is the case, as example:

- with the spbmark package (`https://www.ctan.org/pkg/spbmark`), by Qu Yi, which allows a complete customisation of subscripts, through the `\sub` command;
- with the altsubsup package (`https://www.ctan.org/pkg/altsubsup`), by Julien Labbé, which provides an alternative subscript format, and changes, for this purpose, the catcode of the underscore symbol "_" from 8 (*subscript* catcode category) to 12 (*other* catcode category).

To handle theses cases, the command used for subscript detection can be redefined with `\SetOverArrowsSubscriptCommand`→P. 31. Compatibility with the spbmark package is then obtained by:

```
\SetOverArrowsSubscriptCommand{\sub}
```

In the same way, with the altsubsup package, add:

```
\SetOverArrowsSubscriptCommand{_}
```

after the `\begin{document}` (namely, after the catcode redefinition done by altsubsup).

Alternatively, two package options handle the cases where the catcode of the underscore "_" symbol is changed: `subother`^{→ P. 17} (for catcode 12, or *other*) and `subactive`^{→ P. 17} (for catcode 13, or *active*). Hence, setting the `subother`^{→ P. 17} option is sufficient for compatibility with the altsubsup package (no need of `\SetOverArrowsSubscriptCommand`^{→ P. 31}). Note, that with options `subother`^{→ P. 17} and `subactive`^{→ P. 17}, the command `\TestOverArrow*`^{→ P. 18} may give bad results for kerning test, as defined before the catcode redefinition.

5.2 Package dependencies

The following packages are used by overarrows:

- amsmath
- etoolbox
- pgfkeys
- esvect (unless the option `noesvect`^{→ P. 14} is used)
- old-arrows (when the option `old-arrows`^{→ P. 16} is used)
- tikz (when the `tikz` method or the option `tikz`^{→ P. 16} is used)
- pict2e (when the option `pstarrows`^{→ P. 16} is used)

LaTeX distributions prior to 2020/10/01 must load the xparse package before overarrows.

5.3 Alternatives

esvect package (`https://www.ctan.org/pkg/esvect`), by Eddie Saudrais, provides the fine vector macro `\vv`. This package is loaded by default by overarrows.

letterswitharrows package (`https://www.ctan.org/pkg/letterswitharrows`), by Max Teegen, provides left and right over arrows commands, which can extend to multiple characters.

overrightarrow package (`https://www.ctan.org/pkg/overrightarrow`), by Robin Fairbairns, provides the `\Overrightarrow` which is an amalgam of `\overrightarrow` and `\Rightarrow`.

harpoon package (`https://ctan.org/pkg/harpoon`), by Tobias Kuipers, provides over- and under-harpoon symbol commands.

5.4 Changelog

v1.5
- Provide the command `\baselinebar` → P. 19.
- Redefine `\overbar` → P. 22 and `\underbar` → P. 23 so that they can be nested properly.
- Make always available the lengths `\overarrowthickness` → P. 20 and `\overarrowsmallerthickness` → P. 20.
- Provide the command `\arrowmathstyle` → P. 19.

v1.4 Make commands robust.

v1.3 Bug fix for `esvect` options (see `https://github.com/julienlabbe/latex-packages/issues/2`).

v1.2
- Fix compatibility issues with unicode-math.
- Allow to draw the arrow with PSTricks.
- Make esvect handle all font sizes.
- Allow backslash in command name for `\NewOverArrowCommand` and variants.
- Rewrite starred variant for better performances.

v1.1 Support for non-standard subscripts.

v1.0.1 Bug fix for under* options.

v1.0 Initial version.

6 Implementation

```
\RequirePackage{etoolbox}
```

6.1 Management of options

6.1.1 Declaration of conditionals

```
\newif\ifovar@option@oldarrows@
\newif\ifovar@option@tikz@
\newif\ifovar@option@pstricks@
\newif\ifovar@option@pstarrows@
\newif\ifovar@detectsubscripts@
\newif\ifovar@option@subother@
\newif\ifovar@option@subactive@
\newif\ifovar@option@debug@
```

Following conditionals are for predefined commands.

```
\newif\ifovar@option@overrightarrow@
\newif\ifovar@option@underrightarrow@
\newif\ifovar@option@overleftarrow@
```

```
\newif\ifovar@option@underleftarrow@
\newif\ifovar@option@overleftrightarrow@
\newif\ifovar@option@underleftrightarrow@
\newif\ifovar@option@overrightharpoonup@
\newif\ifovar@option@underrightharpoonup@
\newif\ifovar@option@overrightharpoondown@
\newif\ifovar@option@underrightharpoondown@
\newif\ifovar@option@overleftharpoonup@
\newif\ifovar@option@underleftharpoonup@
\newif\ifovar@option@overleftharpoondown@
\newif\ifovar@option@underleftharpoondown@
\newif\ifovar@option@overbar@
\newif\ifovar@option@underbar@
```

6.1.2 Declaration of options

```
\def\ovar@option@esvect{f}
\DeclareOption{esvect}{\gdef\ovar@option@esvect{f}}
\DeclareOption{noesvect}{\gundef\ovar@option@esvect}
\DeclareOption{esvecta}{\gdef\ovar@option@esvect{a}}
\DeclareOption{esvectb}{\gdef\ovar@option@esvect{b}}
\DeclareOption{esvectc}{\gdef\ovar@option@esvect{c}}
\DeclareOption{esvectd}{\gdef\ovar@option@esvect{d}}
\DeclareOption{esvecte}{\gdef\ovar@option@esvect{e}}
\DeclareOption{esvectf}{\gdef\ovar@option@esvect{f}}
\DeclareOption{esvectg}{\gdef\ovar@option@esvect{g}}
\DeclareOption{esvecth}{\gdef\ovar@option@esvect{h}}
\DeclareOption{old-arrows}{\ovar@option@oldarrows@true}
\DeclareOption{tikz}{\ovar@option@tikz@true}
\DeclareOption{pstricks}{\ovar@option@pstricks@true}
\DeclareOption{pstarrows}{\ovar@option@pstarrows@true}
\DeclareOption{subscripts}{\ovar@detectsubscripts@true}
\DeclareOption{subother}{\ovar@option@subother@true}
\DeclareOption{subactive}{\ovar@option@subactive@true}
\DeclareOption{debug}{\ovar@option@debug@true}
```

Following options are for predefined commands.

```
\DeclareOption{overrightarrow}{\ovar@option@overrightarrow@true}
\DeclareOption{underrightarrow}{\ovar@option@underrightarrow@true}
\DeclareOption{overleftarrow}{\ovar@option@overleftarrow@true}
\DeclareOption{underleftarrow}{\ovar@option@underleftarrow@true}
\DeclareOption{overleftrightarrow}{\ovar@option@overleftrightarrow@true}
\DeclareOption{underleftrightarrow}{\ovar@option@underleftrightarrow@true}
\DeclareOption{overrightharpoonup}{\ovar@option@overrightharpoonup@true}
\DeclareOption{underrightharpoonup}{\ovar@option@underrightharpoonup@true}
\DeclareOption{overrightharpoondown}{\ovar@option@overrightharpoondown@true}
\DeclareOption{underrightharpoondown}{\ovar@option@underrightharpoondown@true}
\DeclareOption{overleftharpoonup}{\ovar@option@overleftharpoonup@true}
\DeclareOption{underleftharpoonup}{\ovar@option@underleftharpoonup@true}
\DeclareOption{overleftharpoondown}{\ovar@option@overleftharpoondown@true}
\DeclareOption{underleftharpoondown}{\ovar@option@underleftharpoondown@true}
\DeclareOption{overbar}{\ovar@option@overbar@true}
\DeclareOption{underbar}{\ovar@option@underbar@true}
```

Following options are for sets of predefined commands.

```
\DeclareOption{overcommands}{%
  \ovar@option@overrightarrow@true
  \ovar@option@overleftarrow@true
  \ovar@option@overleftrightarrow@true
  \ovar@option@overrightharpoonup@true
  \ovar@option@overrightharpoondown@true
  \ovar@option@overleftharpoonup@true
  \ovar@option@overleftharpoondown@true
```

```
  \ovar@option@overbar@true
}
\DeclareOption{undercommands}{%
  \ovar@option@underrightarrow@true
  \ovar@option@underleftarrow@true
  \ovar@option@underleftrightarrow@true
  \ovar@option@underrightharpoonup@true
  \ovar@option@underrightharpoondown@true
  \ovar@option@underleftharpoonup@true
  \ovar@option@underleftharpoondown@true
  \ovar@option@underbar@true
}
\DeclareOption{allcommands}{%
  \ovar@option@overrightarrow@true
  \ovar@option@underrightarrow@true
  \ovar@option@overleftarrow@true
  \ovar@option@underleftarrow@true
  \ovar@option@overleftrightarrow@true
  \ovar@option@underleftrightarrow@true
  \ovar@option@overrightharpoonup@true
  \ovar@option@underrightharpoonup@true
  \ovar@option@overrightharpoondown@true
  \ovar@option@underrightharpoondown@true
  \ovar@option@overleftharpoonup@true
  \ovar@option@underleftharpoonup@true
  \ovar@option@overleftharpoondown@true
  \ovar@option@underleftharpoondown@true
  \ovar@option@overbar@true
  \ovar@option@underbar@true
}
```

6.1.3 Options processing

```
\DeclareOption*{\PackageWarning{overarrows}{Unknown option: '\CurrentOption'}}
\ProcessOptions*
```

6.2 Package dependencies

LaTeX distributions prior to 2020/10/01 must add the xparse package.

etoolbox is loaded at the very start of the package, as `\gundef` is used at options processing.

```
\RequirePackage{amsmath}
```

Option `old-arrows` → P. 16. Configuration of arrows used for predefined commands.

```
\def\ovar@rightarrow{\rightarrow}
\def\ovar@leftarrow{\leftarrow}
\ifovar@option@oldarrows@
  \RequirePackage[old]{old-arrows}
  \def\ovar@rightarrow{\varrightarrow}
  \def\ovar@leftarrow{\varleftarrow}
\fi
```

Option `esvect` → P. 13.

```
\ifdefined\ovar@option@esvect
  \PassOptionsToPackage{\ovar@option@esvect}{esvect}
  \RequirePackage{esvect}
```

Fix font description in `uesvect.fd` to allow any sizes (taken from Enrico Gregorio, `https://tex.stackexchange.com/a/689863/`)

```
  \DeclareFontFamily{U}{esvect}{}
  \DeclareFontShape{U}{esvect}{m}{n}{
    <-5.5> vect5
    <5.5-6.5> vect6
    <6.5-7.5> vect7
    <7.5-8.5> vect8
    <8.5-9.5> vect9
    <9.5-> vect10
  }{}
\fi
```

Option `tikz` → P. 16.

```
\ifovar@option@tikz@
  \RequirePackage{tikz}
  \usetikzlibrary{arrows.meta}
\fi
```

Option `pstricks` → P. 16.

```
\ifovar@option@pstricks@
  \RequirePackage{pstricks-add}
\fi
```

Option `pstarrows` → P. 16.

```
\ifovar@option@pstarrows@
  \RequirePackage[pstarrows]{pict2e}
\fi
```

Add hook rules to apply settings after unicode-math.

```
\DeclareHookRule{begindocument}{overarrows}{after}{unicode-math-luatex}
\DeclareHookRule{begindocument}{overarrows}{after}{unicode-math-xetex}
```

Set `\ovar@auto@middle` and `\ovar@auto@trim@middle`, used by configurations made with `middle config=auto`.

```
\AddToHook{begindocument}[overarrows]
  {%
    \ifdef{\relbareda}
      {%
        \gdef\ovar@auto@middle{\relbareda}
        \gdef\ovar@auto@trim@middle{1}
      }
      {%
        \gdef\ovar@auto@middle{\relbar}
        \gdef\ovar@auto@trim@middle{2.5}
      }%
    \@ifpackageloaded{unicode-math}
      {%
```

Test of `\harrowextender` availability taken from Enrico Gregorio, (`https://tex.stackexchange.com/a/218407/`).

```
        \check@mathfonts
        \iffontchar\textfont\tw@\string"23AF
          \gdef\ovar@auto@middle{\mathrel\harrowextender}
          \gdef\ovar@auto@trim@middle{0}
        \fi
      }
      {}%
  }
```

6.3 Configuration of subscripts detection

`\SetOverArrowsSubscriptCommand`

Sets the subscript command.

```
\newcommand{\SetOverArrowsSubscriptCommand}[1]{\global\let\ovar@subcmd=#1}
```

Initial configuration.

```
\SetOverArrowsSubscriptCommand{_}
```

Option `subother` → P. 17 for *other* (catcode 12) subscript commands.

```
\ifovar@option@subother@
  \begingroup
    \catcode `_=12
    \SetOverArrowsSubscriptCommand{_}%
  \endgroup
\fi
```

Option `subactive` → P. 17 for *active* (catcode 13) subscript commands.

```
\ifovar@option@subactive@
  \begingroup
    \catcode `_=13
    \SetOverArrowsSubscriptCommand{_}%
  \endgroup
\fi
```

6.4 Management of keys

6.4.1 Family declaration and setters

```
\RequirePackage{pgfkeys}
\pgfkeys{overarrows/.is family}
```

\ovar@set

```
\newcommand{\ovar@set}[1]{\pgfqkeys{/overarrows}{#1}}
```

\SetOverArrowsMethod

```
\NewDocumentCommand{\SetOverArrowsMethod}{ s O{fill} m O{} m }{%
  \IfBooleanTF{#1}{%
    \csgdef{ovar@set@#3}{#4\ovar@set{#5}}%
  }{%
    \csgdef{ovar@set@#3}{#4\ovar@set{%
        no stack macro hook/.code={%
          \ovar@set{stack macro/.expanded={%
              \expandafter\expandonce\csname ovar@stack@#2\endcsname%
              {\expandonce\ovar@length@min}%
              {\expandonce\ovar@before@arrow}{\expandonce\ovar@after@arrow}%
            }}%
        },#5}}%
  }%
}
```

6.4.2 Common keys

```
\SetOverArrowsMethod*{common}[\undef{\ovar@macro@stack}\undef{\ovar@macro@arrow}]{%
```

`detect subscripts` → P. 25.

```
  detect subscripts/.is if=ovar@detectsubscripts@,
```

`stack macro` → P. 31 and `arrow macro` → P. 31.

```
  stack macro/.store in=\ovar@macro@stack,
  arrow macro/.store in=\ovar@macro@arrow,
  stack macro/.value required,
  arrow macro/.value required,
```

`no stack macro hook` → P. 31, `no arrow macro hook` → P. 32. These two keys must be redefined by the command `\ovar@set@`⟨*method*⟩.

```
  no stack macro hook/.code={%
    \PackageError{overarrows}{Undefined stack macro}
    {The requested method is perhaps mispelled}
  },
  no arrow macro hook/.code={%
    \PackageError{overarrows}{Undefined arrow macro}
    {The requested method is perhaps mispelled}
  },
```

`min length`[→ P. 23].

```
  min length/.store in=\ovar@length@min,
  min length/.value required,
  min length=0,
```

`before arrow`[→ P. 25], `after arrow`[→ P. 25], `space before arrow`[→ P. 25], `space after arrow`[→ P. 25].

```
  before arrow/.store in=\ovar@before@arrow,
  after arrow/.store in=\ovar@after@arrow,
  before arrow/.value required,
  after arrow/.value required,
  before arrow=\empty,
  after arrow=\empty,
  space before arrow/.code=\pgfkeysalso{before arrow={\kern ##1}},
  space after arrow/.code=\pgfkeysalso{after arrow={\kern ##1}},
```

`shift left`[→ P. 24], `shift right`[→ P. 24], `shift leftright`[→ P. 24], `center arrow`[→ P. 24], `left arrow`[→ P. 24], `right arrow`[→ P. 25].

```
  shift left/.store in=\ovar@shift@left,
  shift right/.store in=\ovar@shift@right,
  shift left/.value required,
  shift right/.value required,
  shift leftright/.code=\pgfkeysalso{%
    shift left=##1, shift right=##1,
  },
  center arrow/.code=\pgfkeysalso{shift leftright=0},
  shift leftright/.value required,
  center arrow/.value forbidden,
  left arrow/.code=\pgfkeysalso{%
    shift left=0, shift right=##1,
  },
  right arrow/.code=\pgfkeysalso{%
    shift left=##1, shift right=0,
  },
  left arrow/.default=2,
  right arrow/.default=2,
  right arrow,
```

`arrow under`[→ P. 24].

```
  arrow under/.is choice,
  arrow under/noconfig/.code={
    \def\ovar@stack@fill{\ovar@stackunder@fill}
    \def\ovar@stack@lens{\ovar@stackunder@lens}
  },
  arrow under/autoconfig/.code={
    \pgfkeysalso{%
      arrow under=noconfig,
      detect subscripts=false,
      before arrow={\kern 1.3\ex@\relax},% like underarrow@ from amsmath
    }
  },
  arrow under/.default=autoconfig,
}
```

6.4.3 Keys for the symb method

```
\SetOverArrowsMethod{symb}[\undef{\ovar@macro@arrowfill}]{%
```

Fill macro.

```
  fill macro/.store in=\ovar@macro@arrowfill,
  fill macro/.value required,
```

Arrow macro.

```
  no arrow macro hook/.code={%
    \ifdef{\ovar@macro@arrowfill}{}{%
      \ovar@set{%
        fill macro/.expanded={%
          \noexpand\ovar@arrow@fill%
          {\expandonce\ovar@shift@left}{\expandonce\ovar@shift@right}%
        }
      }
    }
    \ovar@set{%
      arrow macro/.expanded={%
        \expandonce{\ovar@macro@arrowfill}%
        {\expandonce{\ovar@arrow@start}\expandonce{\ovar@trim@start}}%
        {\expandonce{\ovar@trim@middle}\expandonce{\ovar@arrow@middle}%
          \expandonce{\ovar@trim@middle}}%
        {\expandonce{\ovar@trim@end}\expandonce{\ovar@arrow@end}}%
      }
    }
  },
```

`start`→ P. 26, `middle`→ P. 26, `end`→ P. 26.

```
  start/.store in=\ovar@arrow@start,
  middle/.store in=\ovar@arrow@middle,
  end/.store in=\ovar@arrow@end,
  start/.value required,
  middle/.value required,
  end/.value required,
```

`trim start`→ P. 26, `trim middle`→ P. 26, `trim end`→ P. 26, `trim`→ P. 26, `no trimming`→ P. 26.

```
  trim start/.code={\def\ovar@trim@start{\xjoinrel[##1]}},
  trim middle/.code={\def\ovar@trim@middle{\xjoinrel[##1]}},
  trim end/.code={\def\ovar@trim@end{\xjoinrel[##1]}},
  trim start/.value required,
  trim middle/.value required,
  trim end/.value required,
  trim/.code={\pgfkeysalso{trim start={##1}, trim middle={##1}, trim end={##1}}},
  trim/.value required,
  no trimming/.code={%
    \let\ovar@trim@start\empty
    \let\ovar@trim@middle\empty
    \let\ovar@trim@end\empty
  },
  no trimming/.value forbidden,
```

`middle config`→ P. 26.

```
  middle config/.is choice,
  middle config/.value required,
  middle config/relbar/.code=\pgfkeysalso{%
    middle={\relbar},
    trim middle={2.5},
  },
  middle config/relbareda/.code={%
```

```
    \ifundef{\relbareda}{%
      \PackageWarning{overarrows}{Key 'middle config=relbareda' used,
        \MessageBreak%
        but \protect\relbareda\space is undefined; ignored.
        \MessageBreak%
        Load 'esvect' package, or use 'esvect' option \MessageBreak%
        to remove this warning}
    }{%
      \pgfkeysalso{%
        middle={\relbareda},
        trim middle={1},
      }
    }
  },
  middle config/harrowextender/.code={%
    \pgfkeysalso{%
      middle={\harrowextender},
      trim middle={0},
    }
  },
```

Set `middle config` with (hopefully) a good configuration. It would be better to reuse the previous `middle config` settings, but we have to wait for the `begindocument` hook to know which one to use.

```
  middle config/auto/.code={%
    \pgfkeysalso{%
      middle={\ovar@auto@middle},
      trim middle={\ovar@auto@trim@middle},
    }
  },
```

`amsmath` → P. 27.

```
  amsmath/.is choice,%
  amsmath/mimic/.code=\pgfkeysalso{%
    start={\relbar}, middle={\relbar}, end={\rightarrow},
    trim start=7,
    trim middle=2,
    trim end=7,
    shift leftright=0,
    after arrow={}, before arrow={},
  },
  amsmath/strict/.code=\pgfkeysalso{%
    amsmath=mimic,
    no trimming,
    fill macro={\arrowfill@}, stack macro={\overarrow@},
  },
  amsmath/.default=mimic,
```

`esvect` → P. 27.

```
  esvect/.is choice,%
  esvect/mimic/.code=\pgfkeysalso{%
    start={\relbaredd}, middle={\relbareda}, end={\fldr},
    trim start=1.5,
    trim end=1.5,
    trim middle=0,
    right arrow=2,
    space before arrow=-.7pt,
    space after arrow=-.3pt,
  },
  esvect/strict/.code=\pgfkeysalso{%
    esvect=mimic,
```

```
    no trimming,
    fill macro={\traitfill@}, stack macro={\overvect@},
  },
  esvect/.default=mimic,
```

Initial configuration.

```
  amsmath, middle config=auto, end=\ovar@rightarrow, right arrow,
}
```

6.4.4 Keys for the tikz method

```
\SetOverArrowsMethod[lens]{tikz}[\undef{\ovar@tikz@command}]{%
```

Arrow macro.

```
  no arrow macro hook/.code={%
    \ifdef{\ovar@tikz@command}{}{%
      \pgfkeysgetvalue{/overarrows/path options}{\ovar@tikz@pathoptions}
      \ovar@set{%
        tikz command/.expanded={%
          \noexpand\draw[\expandonce\ovar@tikz@pathoptions]\expandonce\ovar@tikz@path;
        }
      }
    }
    \pgfkeysgetvalue{/overarrows/tikz options}{\ovar@tikz@options}
    \ovar@set{%
      arrow macro/.expanded={%
        $\noexpand\mkern \expandonce{\ovar@shift@left} mu\noexpand\relax$%
        \noexpand\tikz[\expandonce{\ovar@tikz@options}]{\expandonce{\ovar@tikz@command}}%
        $\noexpand\mkern \expandonce{\ovar@shift@right} mu\noexpand\relax$%
      }
    }
  },
```

TikZ parts: `tikz command` → P. 28, `tikz options` → P. 28, `path options` → P. 28, `path` → P. 28.

```
  tikz command/.store in=\ovar@tikz@command,
  tikz options/.initial={x=\overarrowlength, line width=\overarrowthickness},
  path options/.initial={arrows={-Classical TikZ Rightarrow}, cap=round},
  path/.store in=\ovar@tikz@path,
  path={(0,0)--(1,0)},
  tikz command/.value required,
  tikz options/.value required,
  path options/.value required,
  path/.value required,
```

TikZ handy keys: `add path options` → P. 28, `add tikz options` → P. 28, `arrows` → P. 28, `line thickness` → P. 28, `thinner` → P. 28.

```
  add path options/.code=\pgfkeysalso{%
    path options/.append={, ##1}},%
  add tikz options/.code=\pgfkeysalso{%
    tikz options/.append={, ##1}},%
  arrows/.code=\pgfkeysalso{add path options={arrows={##1}}},%
  line thickness/.code=\pgfkeysalso{add path options={line width=##1}},%
  thinner/.code=\pgfkeysalso{line thickness={\overarrowsmallerthickness}},%
  add path options/.value required,%
  add tikz options/.value required,%
  arrows/.value required,%
  line thickness/.value required,%
  thinner/.value forbidden,%
```

Initial configuration.

```
  shift right=-2,
  min length=12,
}
```

6.4.5 Keys for the pstricks method

```
\SetOverArrowsMethod[lens]{pstricks}{%
```

Arrow macro.

```
  no arrow macro hook/.code={%
    \ovar@set{%
      arrow macro/.expanded={%
        $\noexpand\mkern \expandonce{\ovar@shift@left} mu\noexpand\relax$%
        \noexpand\begin{pspicture}\expandonce{\ovar@pstricks@geometry}%
          \noexpand\psset{linewidth=\expandonce{\ovar@pstricks@linethickness}}%
          \noexpand\psset{\expandonce{\ovar@pstricks@psset}}%
          \expandonce{\ovar@pstricks@command}%
        \noexpand\end{pspicture}%
        $\noexpand\mkern \expandonce{\ovar@shift@right} mu\noexpand\relax$%
      }
    }
  },
```

Pstricks parts: `pstricks command` → P. 29, `psset` → P. 29, `geometry` → P. 29, `line thickness` → P. 29.

```
  pstricks command/.store in=\ovar@pstricks@command,
  psset/.store in=\ovar@pstricks@psset,
  geometry/.store in=\ovar@pstricks@geometry,
  line thickness/.store in=\ovar@pstricks@linethickness,
  pstricks command/.value required,
  psset/.value required,
  geometry/.value required,
  line thickness/.value required,
```

Pstricks handy key: `arrow` → P. 29, `thinner` → P. 29.

```
  arrow/.style={pstricks command={\psline{##1}(0,0)(\overarrowlength,0)}},%
  arrow/.value required,%
  thinner/.style={line thickness={\overarrowsmallerthickness}},%
  thinner/.value forbidden,%
```

Initial configuration.

```
  shift right=-2,
  min length=12,
  geometry={(0,-0.5ex)(\overarrowlength,0.5ex)},%
  line thickness={\overarrowthickness},%
  arrow={->},%
  psset={},%
}
```

6.4.6 Keys for the picture method

```
\SetOverArrowsMethod[lens]{picture}{%
```

Arrow macro.

```
  no arrow macro hook/.code={%
    \ovar@set{%
      arrow macro/.expanded={%
        $\noexpand\mkern \expandonce{\ovar@shift@left} mu\noexpand\relax$%
        \noexpand\begin{picture}\expandonce{\ovar@picture@geometry}%
          \noexpand\linethickness{\expandonce{\ovar@picture@linethickness}}%
          \expandonce{\ovar@picture@command}%
          \noexpand\end{picture}%
```

```
        $\noexpand\mkern \expandonce{\ovar@shift@right} mu\noexpand\relax$%
      }
    }
  },
```

Picture parts: `picture command`→ P. 30, `geometry`→ P. 30, `line thickness`→ P. 30.

```
  picture command/.store in=\ovar@picture@command,
  geometry/.store in=\ovar@picture@geometry,
  line thickness/.store in=\ovar@picture@linethickness,
  picture command/.value required,
  geometry/.value required,
  line thickness/.value required,
```

Picture handy key: `thinner`→ P. 30.

```
  thinner/.code=\pgfkeysalso{line thickness={\overarrowsmallerthickness}},
```

Initial configuration.

```
  shift right=-2,
  min length=18,
  geometry={(\overarrowlength,1ex)(0,-0.5ex)},%
  line thickness={\overarrowthickness},%
  picture command={\put(0,0){\vector(1,0){\overarrowlength}}},%
}
```

6.5 Commands

6.5.1 Macros for symbols assemblage

`\xjoinrel`

```
\ifdef{\xjoinrel}{%
  \PackageWarning{overarrows}{Command \protect\xjoinrel\space already defined.
    \MessageBreak%
    Previous definition will be overridden}
}{}
```

Use a default value of `3.5 mu`, as recommended by Enrico Gregorio (see `https://tex.stackexchange.com/a/471736`). `\joinrel` uses a value of 3 mu.

```
\DeclareRobustCommand{\xjoinrel}[1][3.5]{\mathrel{\mkern-#1mu}}
```

`\smallermathstyle`

```
\newcommand*{\smallermathstyle}{%
  \mathchoice{\scriptstyle}{\scriptstyle}{\scriptscriptstyle}{}
}
```

`\arrowmathstyle`

Math styles for arrows commands. `#1`: math style

`\ovar@save@mathstyle`

```
\def\ovar@save@mathstyle#1{%
  \let\arrowmathstyle#1%
}%
```

`\ovar@bar`

Draw a zero height horizontal bar aligned on the baseline.
`#1`: math style

```
\def\ovar@bar#1{%
  \raisebox
    {\dimeval{-0.5\height+0.5\depth}} [0pt] [0pt]
    {$\m@th #1\vphantom{+}\std@minus$}%
}
```

`\baselinebar`

Draw a zero height horizontal bar aligned on the baseline with the current math style (use `\arrowmathstyle`, if defined).

```
\NewDocumentCommand{\baselinebar} { } {%
  \mathrel {%
    \ifdef \arrowmathstyle
      {\ovar@bar\arrowmathstyle}
```

```
      {\mathpalette\ovar@bar{}}%
  }%
}
```

\ovar@arrow@fill

Macro used for default `fill macro` → P. 32.
#1: left shift
#2: right shift
#3: arrow start
#4: arrow middle
#5: arrow end
#6: math style

```
\def\ovar@arrow@fill#1#2#3#4#5#6{%
  $\m@th\thickmuskip0mu\medmuskip\thickmuskip\thinmuskip\thickmuskip\relax%
  \mkern #1 mu\relax#6#3%
  \cleaders\hbox{$#6#4$}\hfill%
  #5\mkern #2 mu\relax$%
}
```

6.5.2 Macros for fixed length arrows

Lengths declaration.

```
\newlength{\overarrowlength}
\newlength{\overarrowthickness}
\newlength{\overarrowsmallerthickness}
\newlength{\ovar@tempdim}
```

\ovar@set@arrowlength

Sets `\overarrowlength` → P. 20.
#1: min length, in math units
#2: math style
#3: content

```
\def\ovar@set@arrowlength#1#2#3{%
  \settowidth{\ovar@tempdim}{$\m@th#2\mskip #1 mu\relax$}%
  \settowidth{\overarrowlength}{$\m@th#2#3$}%
  \ifdim \overarrowlength < \ovar@tempdim \overarrowlength=\ovar@tempdim\fi%
}
```

\ovar@set@arrowthickness

Sets `\overarrowthickness` → P. 20 and `\overarrowsmallerthickness` → P. 20.

\ovar@set@arrowthickness@UM@lua

#1: math style

Set to the default rule thickness of the current math style, normaly given by `\fontdimen` 8 family 3. With unicode-math, use instead:

- `\fontdimen` 54 family 2 with XeTeX,
- `\Umathoverbarrule` with LuaTex.

```
\def\ovar@rulethickness@fontdimen{8}
\def\ovar@rulethickness@family{3}
\def\ovar@set@arrowthickness#1{%
  \ifx#1\displaystyle%
    \overarrowthickness =
      \fontdimen \ovar@rulethickness@fontdimen  \textfont \ovar@rulethickness@family%
    \overarrowsmallerthickness =
      \fontdimen \ovar@rulethickness@fontdimen \scriptfont \ovar@rulethickness@family%
  \else\ifx#1\textstyle%
    \overarrowthickness =
      \fontdimen \ovar@rulethickness@fontdimen \textfont \ovar@rulethickness@family%
    \overarrowsmallerthickness =
      \fontdimen \ovar@rulethickness@fontdimen \scriptfont \ovar@rulethickness@family%
```

```
  \else\ifx#1\scriptstyle%
    \overarrowthickness =
      \fontdimen \ovar@rulethickness@fontdimen \scriptfont \ovar@rulethickness@family%
    \overarrowsmallerthickness =
      \fontdimen \ovar@rulethickness@fontdimen \scriptscriptfont \ovar@rulethickness@family%
  \else%
    \overarrowthickness =
      \fontdimen \ovar@rulethickness@fontdimen \scriptscriptfont \ovar@rulethickness@family%
    \overarrowsmallerthickness = \overarrowthickness%
  \fi\fi\fi%
}
```

unicode-math with LuaTeX version.

```
\def\ovar@set@arrowthickness@UM@lua#1{%
  \overarrowthickness = \Umathoverbarrule #1
  \ifx#1\displaystyle%
    \overarrowsmallerthickness = \Umathoverbarrule \textstyle%
  \else\ifx#1\textstyle%
    \overarrowsmallerthickness = \Umathoverbarrule \scriptstyle%
  \else%
    \overarrowsmallerthickness = \Umathoverbarrule \scriptscriptstyle%
  \fi\fi%
}
```

Test which version to use.

```
\AddToHook{begindocument}[overarrows]
  {%
    \@ifpackageloaded{unicode-math-luatex}
      {%
        \global\let\ovar@set@arrowthickness\ovar@set@arrowthickness@UM@lua
      }
      {%
        \@ifpackageloaded{unicode-math-xetex}
          {%
            \gdef\ovar@rulethickness@fontdimen{54}
            \gdef\ovar@rulethickness@family{2}
          }
          {}
      }
  }
```

6.5.3 Stack macros

\ovar@stackover@@

\ovar@stackunder@@

Bases of all stack macros.
#1: min length, in math units
#2: vertical mode material before arrow
#3: vertical mode material after arrow
#4: arrow
#5: math style
#6: content

```
\def\ovar@stackover@@#1#2#3#4#5#6{%
  \ovar@save@mathstyle{#5}%
  \ovar@set@arrowthickness{#5}%
  \vbox{%
    \ialign{##\crcr%
      $#5\mskip #1 mu\relax$\crcr%
      \noalign{#2\nointerlineskip}#4\crcr%
      \noalign{#3\nointerlineskip}%
      $\m@th\hfil#5#6\hfil$\crcr%
```

```
    }%
  }%
}
\def\ovar@stackunder@@#1#2#3#4#5#6{%
  \ovar@save@mathstyle{#5}%
  \ovar@set@arrowthickness{#5}%
  \vtop{%
    \ialign{##\crcr%
      $\m@th\hfil#5#6\hfil$\crcr%
      \noalign{#2\nointerlineskip}#4\crcr%
      \noalign{#3\nointerlineskip}%
      $#5\mskip #1 mu\relax$\crcr%
    }%
  }%
}
```

`\ovar@stackover@`

`\ovar@stackunder@`

Stack macros without min arrow length.
#1: vertical mode material before arrow
#2: vertical mode material after arrow
#3: arrow macro
#4: math style
#5: content

```
\def\ovar@stackover@#1#2#3#4#5{\ovar@stackover@@{0}{#1}{#2}{#3}{#4}{#5}}
\def\ovar@stackunder@#1#2#3#4#5{\ovar@stackunder@@{0}{#1}{#2}{#3}{#4}{#5}}
```

`\ovar@stackover@fill`

`\ovar@stackunder@fill`

`\ovar@stack@fill`

Stack macros for extensible arrows.
#1: min length, in math units
#2: vertical mode material before arrow
#3: vertical mode material after arrow
#4: arrow filler macro
#5: math style
#6: content

```
\def\ovar@stackover@fill#1#2#3#4#5#6{\ovar@stackover@@{#1}{#2}{#3}{#4#5}{#5}{#6}}
\def\ovar@stackunder@fill#1#2#3#4#5#6{\ovar@stackunder@@{#1}{#2}{#3}{#4#5}{#5}{#6}}
```

`\ovar@stack@fill` matches the macro `\ovar@stackover@fill` by default, or `\ovar@stackunder@fill` with `arrow under`→ P. 24.

```
\def\ovar@stack@fill{\ovar@stackover@fill}
```

`\ovar@stackover@lens`

`\ovar@stackunder@lens`

`\ovar@stack@lens`

Stack macros for fixed-length arrows (these call `\ovar@set@arrowlength` and `\ovar@set@arrowthickness`).
#1: min length, in math units
#2: vertical mode material before arrow
#3: vertical mode material after arrow
#4: arrow content macro
#5: math style
#6: content

```
\def\ovar@stackover@lens#1#2#3#4#5#6{%
  \ovar@set@arrowlength{#1}{#5}{#6}%
  \ovar@stackover@{#2}{#3}{#4}{#5}{#6}%
}
\def\ovar@stackunder@lens#1#2#3#4#5#6{%
  \ovar@set@arrowlength{#1}{#5}{#6}%
  \ovar@stackunder@{#2}{#3}{#4}{#5}{#6}%
}
```

`\ovar@stack@lens` matches the macro `\ovar@stackover@lens` by default, or `\ovar@stackunder@lens` with `arrow under`→ P. 24.

```
\def\ovar@stack@lens{\ovar@stackover@lens}
```

6.5.4 Macro for commands creation

In the initial version, the commands names must be given as csname (without backslash). To harmonize the syntax with standard `\NewDocumentCommand`, define an argument processor so that both `\NewOverArrowCommand{\myarrow}` and `\NewOverArrowCommand{myarrow}` are accepted.

```
\ExplSyntaxOn
\cs_new_protected:Npn \__overarrows_processor_strip_escape_char:n #1
  {
     \regex_match:nnTF { ^\cC. } { #1 }
     { \tl_set:Nx \ProcessedArgument { \cs_to_str:N #1 } }
     { \tl_set:Nx \ProcessedArgument { #1 } }
  }
\cs_new_eq:NN \ovar@cmdname@processor \__overarrows_processor_strip_escape_char:n
\ExplSyntaxOff
```

`\DeclareOverArrowCommand`

```
\NewDocumentCommand{\DeclareOverArrowCommand}{
 O{symb} >{\ovar@cmdname@processor} m m
}{%
  \begingroup
  \ovar@set@common
  \ifcsdef{ovar@set@#1}{%
    \csuse{ovar@set@#1}
  }{%
    \PackageError{overarrows}{Unknown method #1}
    {Try with 'symb', 'tikz', 'pstriks' or 'picture'}
  }
  \ovar@set{#3}
  \ifdef{\ovar@macro@arrow}{}{%
    \ovar@set{no arrow macro hook}
  }
  \ifdef{\ovar@macro@stack}{}{%
    \ovar@set{no stack macro hook}
  }
  \csxdef{ovar@#2@normal}{%
    \noexpand\mathpalette{%
      \expandonce{\ovar@macro@stack}{\expandonce{\ovar@macro@arrow}}%
    }
  }
  \csxdef{ovar@#2@starred}{%
    \noexpand\mathpalette{%
      \noexpand\ovar@starversion{%
        \expandonce{\ovar@macro@stack}{\expandonce{\ovar@macro@arrow}}%
      }
    }
  }
  \ifovar@option@debug@
  \PackageInfo{overarrows}{%
    Meaning of \protect\ovar@#2@normal\MessageBreak
    used for \@backslashchar#2:\MessageBreak%
      \expandafter\meaning\csname ovar@#2@normal\endcsname}
  \fi
```

Expand `\ifovar@detectsubscripts@` before closing the group, then define the command.

```
  \expandafter%
  \endgroup
  \ifovar@detectsubscripts@%
```

```
  \csgdef{ovar@#2@auto}##1{%
    \@ifnextchar \ovar@subcmd {%
      \csuse{ovar@#2@starred}{##1}%
    }{%
      \csuse{ovar@#2@normal}{##1}%
    }%
  }
  \expandafter\DeclareDocumentCommand\csname #2\endcsname { s }{%
    \IfBooleanTF{##1}{\csuse{ovar@#2@starred}}{\csuse{ovar@#2@auto}}%
  }%
  \else
  \expandafter\DeclareDocumentCommand\csname #2\endcsname { s }{%
    \IfBooleanTF{##1}{\csuse{ovar@#2@starred}}{\csuse{ovar@#2@normal}}%
  }%
  \fi
}
```

\ProvideOverArrowCommand

```
\NewDocumentCommand{\ProvideOverArrowCommand}{
 O{symb} >{\ovar@cmdname@processor} m m
}{%
  \ifcsdef{#2}{}{
    \DeclareOverArrowCommand[#1]{#2}{#3}
  }
}
```

\NewOverArrowCommand

```
\NewDocumentCommand{\NewOverArrowCommand}{
 O{symb} >{\ovar@cmdname@processor} m m
}{%
  \ifcsdef{#2}{%
    \PackageError{overarrows}{Command \csname #2\endcsname already defined}%
    {You have used \protect\NewOverArrowCommand\space with a command that
      already has a definition. \MessageBreak%
      Choose another name, or use instead \protect\DeclareOverArrowCommand.}
  }{%
    \DeclareOverArrowCommand[#1]{#2}{#3}
  }
}
```

\RenewOverArrowCommand

```
\NewDocumentCommand{\RenewOverArrowCommand}{
 O{symb} >{\ovar@cmdname@processor} m m
}{%
  \ifcsundef{#2}{%
    \PackageError{overarrows}{Command  \csname #2\endcsname undefined}%
    {You have used \protect\RenewOverArrowCommand\space with a command that was
      never defined. \MessageBreak%
      Check the requested name, or use instead \protect\NewOverArrowCommand.}
  }{%
    \DeclareOverArrowCommand[#1]{#2}{#3}
  }
}
```

6.5.5 Starred variant

\ovar@starversion

#1: definition (stack macro + arrow macro)
#2: math style
#3: content

```
\newsavebox\ovar@tempbox
\def\ovar@starversion#1#2#3{%
  \sbox{\ovar@tempbox}{$\m@th #1#2{#3}$}%
  \usebox{\ovar@tempbox}%
```

Remove the extra space added by the arrow.

```
  \settowidth{\ovar@tempdim}{$\m@th #2{#3}$}%
  \kern\dimeval{0.5\ovar@tempdim - 0.5\wd\ovar@tempbox}%
}
```

6.5.6 \vv vector command

\vv

Backup and redefinition of esvect \vv → P. 21 vector command.

\esvectvv

```
\ifdefined\ovar@option@esvect
  \let\esvectvv\vv
  \undef\vv
  \NewOverArrowCommand{\vv}{esvect, middle config = relbareda}
\fi
```

6.5.7 Predefined commands

Declare predefined commands after unicode-math settings.

```
\AddToHook{begindocument}[overarrows]
 {
```

\overrightarrow

```
   \ifovar@option@overrightarrow@
     \DeclareOverArrowCommand{\overrightarrow}{%
       amsmath, middle config=relbar,
       end=\ovar@rightarrow,
       right arrow,
     }
   \fi
```

\underrightarrow

```
   \ifovar@option@underrightarrow@
     \DeclareOverArrowCommand{\underrightarrow}{%
       amsmath, middle config=relbar,
       end=\ovar@rightarrow,
       right arrow,
       arrow under,
     }
   \fi
```

\overleftarrow

```
   \ifovar@option@overleftarrow@
     \DeclareOverArrowCommand{\overleftarrow}{%
       amsmath, middle config=relbar,
       start=\ovar@leftarrow,
       end=\relbar,
       left arrow,
     }
   \fi
```

\underleftarrow

```
   \ifovar@option@underleftarrow@
     \DeclareOverArrowCommand{\underleftarrow}{%
       amsmath, middle config=relbar,
       start=\ovar@leftarrow,
       end=\relbar,
       left arrow,
       arrow under,
     }
   \fi
```

\overleftrightarrow

```
   \ifovar@option@overleftrightarrow@
     \DeclareOverArrowCommand{\overleftrightarrow}{%
       amsmath, middle config=relbar,
       start=\ovar@leftarrow,
       end=\ovar@rightarrow,
```

```
        center arrow,
      }
    \fi
\underleftrightarrow
    \ifovar@option@underleftrightarrow@
      \DeclareOverArrowCommand{\underleftrightarrow}{%
        amsmath, middle config=relbar,
        start=\ovar@leftarrow,
        end=\ovar@rightarrow,
        center arrow,
        arrow under,
      }
    \fi
\overrightharpoonup
    \ifovar@option@overrightharpoonup@
      \DeclareOverArrowCommand{\overrightharpoonup}{%
        amsmath, middle config=relbar,
        end=\rightharpoonup,
        right arrow,
      }
    \fi
\underrightharpoonup
    \ifovar@option@underrightharpoonup@
      \DeclareOverArrowCommand{\underrightharpoonup}{%
        amsmath, middle config=relbar,
        end=\rightharpoonup,
        right arrow,
        arrow under,
      }
    \fi
\overrightharpoondown
    \ifovar@option@overrightharpoondown@
      \DeclareOverArrowCommand{\overrightharpoondown}{%
        amsmath, middle config=relbar,
        end=\rightharpoondown,
        right arrow,
      }
    \fi
\underrightharpoondown
    \ifovar@option@underrightharpoondown@
      \DeclareOverArrowCommand{\underrightharpoondown}{%
        amsmath, middle config=relbar,
        end=\rightharpoondown,
        right arrow,
        arrow under,
      }
    \fi
\overleftharpoonup
    \ifovar@option@overleftharpoonup@
      \DeclareOverArrowCommand{\overleftharpoonup}{%
        amsmath, middle config=relbar,
        start=\leftharpoonup,
        end=\relbar,
        left arrow,
      }
    \fi
\underleftharpoonup
    \ifovar@option@underleftharpoonup@
      \DeclareOverArrowCommand{\underleftharpoonup}{%
        amsmath, middle config=relbar,
        start=\leftharpoonup,
        end=\relbar,
        left arrow,
        arrow under,
      }
    \fi
```

\overleftharpoondown
```
    \ifovar@option@overleftharpoondown@
      \DeclareOverArrowCommand{\overleftharpoondown}{%
        amsmath, middle config=relbar,
        start=\leftharpoondown,
        end=\relbar,
        left arrow,
      }
    \fi
```
\underleftharpoondown
```
    \ifovar@option@underleftharpoondown@
      \DeclareOverArrowCommand{\underleftharpoondown}{%
        amsmath, middle config=relbar,
        start=\leftharpoondown,
        end=\relbar,
        left arrow,
        arrow under,
      }
    \fi
```
\overbar
```
    \ifovar@option@overbar@
      \DeclareOverArrowCommand{\overbar}{%
        amsmath, middle={\baselinebar},
        start={\baselinebar}, end={\baselinebar},
        space before arrow={0.5\overarrowthickness},
        space after arrow={3.5\overarrowthickness}
      }
    \fi
```
\underbar
```
    \ifovar@option@underbar@
      \DeclareOverArrowCommand{\underbar}{%
        amsmath, middle={\baselinebar},
        start={\baselinebar}, end={\baselinebar},
        arrow under=noconfig,
        space before arrow={3.5\overarrowthickness},
        space after arrow={0.5\overarrowthickness}
      }
    \fi
```
End of `\AddToHook{begindocument}` hook.
```
  }
```

6.5.8 Test macros

\ovar@testmathstyles

Tabular containing the output of a command for the four math styles and different patterns.

```
\newcommand{\ovar@testmathstyles}[2][]{
  \begingroup
  \newcommand*{\ovar@row@teststyle}[1]{%
    $\displaystyle ##1$
    & $\textstyle ##1$
    & $\scriptstyle ##1$
    & $\scriptscriptstyle ##1$
    \\
  }
  \renewcommand*{\arraystretch}{1.5}
  \begin{tabular*}{0.95\linewidth}{@{\extracolsep{\fill}} cccc}
    \hline
    \footnotesize\texttt{\texttt{\textbackslash displaystyle}}
    & \footnotesize\texttt{\texttt{\textbackslash textstyle}}
    & \footnotesize\texttt{\texttt{\textbackslash scriptstyle}}
    & \footnotesize\texttt{\texttt{\textbackslash scriptscriptstyle}}
```

```
    \\
    \hline
    \ovar@row@teststyle{\csuse{#2}{v}}
    \ovar@row@teststyle{\csuse{#2}{AB}}
    \ovar@row@teststyle{\csuse{#2}{\mathrm{grad}}}
    \ovar@row@teststyle{\csuse{#2}{my~long~vector}}
    \IfValueT{#1}{\ovar@row@teststyle{\csuse{#2}{#1}}}
    \hline
  \end{tabular*}
  \endgroup
}
```

\ovar@testkerning

```
\begingroup
\ifovar@option@subother@  \catcode `_=12 \fi
\ifovar@option@subactive@ \catcode `_=13 \fi
\gdef\ovar@testkerning#1{%
  \begin{displaymath}
    #1{t}_{#1{u}_{#1{v}}}
    \qquad
    #1{\imath}_0
    \qquad
    #1{v}
    = #1{v}_x + #1{v}_y + #1{v}_z
    = v_x #1{\imath} + v_y #1{\jmath} + v_z #1{k}
  \end{displaymath}
}
\endgroup
```

\TestOverArrow

```
\NewDocumentCommand{\TestOverArrow}{
  s o >{\ovar@cmdname@processor} m
}{%
  \ifcsdef{#3}{}{%
    \PackageWarning{overarrows}{Unknown name '#3' passed to
      \protect\TestOverArrow}
  }
  \IfBooleanTF{#1}{%
    \noindent\framebox{%
      \begin{minipage}{0.95\linewidth}
        \centering
        \noindent\textbf{\large%
          Test of \texttt{\textbackslash#3} and \texttt{\textbackslash#3*} macros}
        \bigskip\par
        \textbf{\texttt{\textbackslash#3} for different math styles}
        \smallskip\par
        \ovar@testmathstyles[#2]{#3}%
        \bigskip\par
        \textbf{\texttt{\textbackslash#3} kerning}
        \ovar@testkerning{\csuse{#3}}
        \textbf{\texttt{\textbackslash#3*} kerning}
        \ovar@testkerning{\csuse{#3}*}
      \end{minipage}%
    }\bigskip\par
  }{%
    \ovar@testmathstyles[#2]{#3}%
  }
}
```

Index

Entries listed in the categories “commands”, “lengths”, and “internal macros” also include references to package implementation.

Package options
- allcommands, 15
- debug, 17
- esvect, 13
- esvecta, 14
- esvectb, 14
- esvectc, 14
- esvectd, 14
- esvecte, 14
- esvectf, 14
- esvectg, 14
- esvecth, 14
- noesvect, 14
- old-arrows, 16
- overbar, 15
- overcommands, 15
- overleftarrow, 15
- overleftharpoondown, 15
- overleftharpoonup, 15
- overleftrightarrow, 15
- overrightarrow, 15
- overrightharpoondown, 15
- overrightharpoonup, 15
- pstarrows, 16
- pstricks, 16
- subactive, 17
- subother, 17
- subscripts, 17
- tikz, 16
- underbar, 16
- undercommands, 15
- underleftarrow, 15
- underleftharpoondown, 16
- underleftharpoonup, 16
- underleftrightarrow, 15
- underrightarrow, 15
- underrightharpoondown, 16
- underrightharpoonup, 16

add path options key, 28
add tikz options key, 28
after arrow key, 25
allcommands package option, 15
amsmath key, 27
arrow key, 29
arrow macro key, 31
arrow under key, 24
\arrowmathstyle, 19
arrows key, 28

\baselinebar, 19
before arrow key, 25

center arrow key, 24
Commands
- \arrowmathstyle, 19, 44
- \baselinebar, 19, 44, 52
- \DeclareOverArrowCommand, 17, 48–52
- \esvectvv, 21, 50
- \NewOverArrowCommand, 17, 49, 50
- \overbar, 22, 35, 52
- \overleftarrow, 22, 35, 50
- \overleftharpoondown, 22, 35, 52
- \overleftharpoonup, 22, 35, 51
- \overleftrightarrow, 22, 35, 50
- \overrightarrow, 21, 35, 50
- \overrightharpoondown, 22, 35, 51
- \overrightharpoonup, 22, 35, 51
- \ProvideOverArrowCommand, 17, 49
- \RenewOverArrowCommand, 17, 49
- \SetOverArrowsMethod, 31, 38, 40, 42, 43
- \SetOverArrowsMethod*, 31
- \SetOverArrowsSubscriptCommand, 31, 38
- \smallermathstyle, 19, 44
- \TestOverArrow, 18, 53
- \TestOverArrow*, 18
- \underbar, 23, 35, 52
- \underleftarrow, 22, 35, 50
- \underleftharpoondown, 23, 35, 52
- \underleftharpoonup, 23, 35, 51
- \underleftrightarrow, 22, 35, 51
- \underrightarrow, 22, 35, 50
- \underrightharpoondown, 23, 35, 51
- \underrightharpoonup, 23, 35, 51
- \vv, 21, 50
- \vv*, 21
- \xjoinrel, 19, 40, 44

debug package option, 17

\DeclareOverArrowCommand, 17
detect subscripts key, 25

end key, 26
esvect key, 27
esvect package option, 13
esvecta package option, 14
esvectb package option, 14
esvectc package option, 14
esvectd package option, 14
esvecte package option, 14
esvectf package option, 14
esvectg package option, 14
esvecth package option, 14
\esvectvv, 21

fill macro key, 32

geometry key, 29, 30

Internal macros
\ifovar@detectsubscripts@, 34, 48
\ifovar@option@debug@, 34, 48
\ifovar@option@oldarrows@, 34, 36
\ifovar@option@overbar@, 35, 52
\ifovar@option@overleftarrow@, 34, 50
\ifovar@option@overleftharpoondown@, 35, 52
\ifovar@option@overleftharpoonup@, 35, 51
\ifovar@option@overleftrightarrow@, 35, 50
\ifovar@option@overrightarrow@, 34, 50
\ifovar@option@overrightharpoondown@, 35, 51
\ifovar@option@overrightharpoonup@, 35, 51
\ifovar@option@pstarrows@, 34, 37
\ifovar@option@pstricks@, 34, 37
\ifovar@option@subactive@, 34, 38, 53
\ifovar@option@subother@, 34, 38, 53
\ifovar@option@tikz@, 34, 37
\ifovar@option@underbar@, 35, 52
\ifovar@option@underleftarrow@, 35, 50
\ifovar@option@underleftharpoondown@, 35, 52
\ifovar@option@underleftharpoonup@, 35, 51
\ifovar@option@underleftrightarrow@, 35, 51
\ifovar@option@underrightarrow@, 34, 50
\ifovar@option@underrightharpoondown@, 35, 51
\ifovar@option@underrightharpoonup@, 35, 51
\ovar@after@arrow, 38, 39
\ovar@arrow@end, 40
\ovar@arrow@fill, 40, 45
\ovar@arrow@middle, 40
\ovar@arrow@start, 40
\ovar@auto@middle, 37, 41
\ovar@auto@trim@middle, 37, 41
\ovar@bar, 44, 45
\ovar@before@arrow, 38, 39
\ovar@cmdname@processor, 48, 49, 53
\ovar@leftarrow, 36, 50, 51
\ovar@length@min, 38, 39
\ovar@macro@arrow, 38, 48
\ovar@macro@arrowfill, 40
\ovar@macro@stack, 38, 48
\ovar@option@esvect, 35, 36, 50
\ovar@picture@command, 43, 44
\ovar@picture@geometry, 43, 44
\ovar@picture@linethickness, 43, 44
\ovar@pstricks@command, 43
\ovar@pstricks@geometry, 43
\ovar@pstricks@linethickness, 43
\ovar@pstricks@psset, 43
\ovar@rightarrow, 36, 42, 50, 51
\ovar@row@teststyle, 52, 53
\ovar@rulethickness@family, 45, 46
\ovar@rulethickness@fontdimen, 45, 46
\ovar@save@mathstyle, 44, 46, 47
\ovar@set, 38, 40, 42, 43, 48
\ovar@set@arrowlength, 45, 47
\ovar@set@arrowthickness, 45–47
\ovar@set@arrowthickness@UM@lua, 46
\ovar@set@common, 48
\ovar@shift@left, 39, 40, 42, 43
\ovar@shift@right, 39, 40, 42–44
\ovar@stack@fill, 39, 47
\ovar@stack@lens, 39, 48
\ovar@stackover@, 47

\ovar@stackover@@, 46, 47
\ovar@stackover@fill, 47
\ovar@stackover@lens, 47, 48
\ovar@stackunder@, 47
\ovar@stackunder@@, 47
\ovar@stackunder@fill, 39, 47
\ovar@stackunder@lens, 39, 47
\ovar@starversion, 48, 49
\ovar@subcmd, 38, 49
\ovar@tempbox, 49, 50
\ovar@tempdim, 45, 50
\ovar@testkerning, 53
\ovar@testmathstyles, 52, 53
\ovar@tikz@command, 42
\ovar@tikz@options, 42
\ovar@tikz@path, 42
\ovar@tikz@pathoptions, 42
\ovar@trim@end, 40
\ovar@trim@middle, 40
\ovar@trim@start, 40

Keys
add path options, 28
add tikz options, 28
after arrow, 25
amsmath, 27
arrow, 29
arrow macro, 31
arrow under, 24
arrows, 28
before arrow, 25
center arrow, 24
detect subscripts, 25
end, 26
esvect, 27
fill macro, 32
geometry, 29, 30
left arrow, 24
line thickness, 28–30
middle, 26
middle config, 26
min length, 23
no arrow macro hook, 32
no stack macro hook, 31
no trimming, 26
path, 28
path options, 28
picture command, 30
psset, 29
pstricks command, 29
right arrow, 25
shift left, 24
shift leftright, 24
shift right, 24
space after arrow, 25
space before arrow, 25
stack macro, 31
start, 26
thinner, 28–30
tikz command, 28
tikz options, 28
trim, 26
trim end, 26
trim middle, 26
trim start, 26

left arrow key, 24
Lengths
\overarrowlength, 20, 42–45
\overarrowsmallerthickness, 20, 42–46
\overarrowthickness, 20, 42–46, 52
line thickness key, 28–30

middle key, 26
middle config key, 26
min length key, 23

\NewOverArrowCommand, 17
no arrow macro hook key, 32
no stack macro hook key, 31
no trimming key, 26
noesvect package option, 14

old-arrows package option, 16
\overarrowlength length, 20
\overarrowsmallerthickness length, 20
\overarrowthickness length, 20
\overbar, 22
overbar package option, 15
overcommands package option, 15
\overleftarrow, 22
overleftarrow package option, 15
\overleftharpoondown, 22
overleftharpoondown package option, 15
\overleftharpoonup, 22
overleftharpoonup package option, 15
\overleftrightarrow, 22
overleftrightarrow package option, 15
\overrightarrow, 21
overrightarrow package option, 15
\overrightharpoondown, 22
overrightharpoondown package option, 15
\overrightharpoonup, 22
overrightharpoonup package option, 15

path key, 28
path options key, 28
picture command key, 30
\ProvideOverArrowCommand, 17
psset key, 29
pstarrows package option, 16
pstricks package option, 16
pstricks command key, 29

\RenewOverArrowCommand, 17
right arrow key, 25

\SetOverArrowsMethod, 31
\SetOverArrowsMethod*, 31
\SetOverArrowsSubscriptCommand, 31
shift left key, 24
shift leftright key, 24
shift right key, 24
\smallermathstyle, 19
space after arrow key, 25
space before arrow key, 25
stack macro key, 31
start key, 26
subactive package option, 17
subother package option, 17
subscripts package option, 17

\TestOverArrow, 18
\TestOverArrow*, 18
thinner key, 28–30
tikz package option, 16
tikz command key, 28
tikz options key, 28
trim key, 26
trim end key, 26
trim middle key, 26
trim start key, 26

\underbar, 23
underbar package option, 16
undercommands package option, 15
\underleftarrow, 22
underleftarrow package option, 15
\underleftharpoondown, 23
underleftharpoondown package option, 16
\underleftharpoonup, 23
underleftharpoonup package option, 16
\underleftrightarrow, 22
underleftrightarrow package option, 15
\underrightarrow, 22
underrightarrow package option, 15
\underrightharpoondown, 23
underrightharpoondown package option, 16
\underrightharpoonup, 23
underrightharpoonup package option, 16

\vv, 21
\vv*, 21

\xjoinrel, 19

Change History

v1.0
General: Initial version 1
v1.0.1
General: Bug fix for under* options 35
v1.1
General: Support for non-standard subscripts 37, 49, 53
v1.2
General: Add option pstricks 35
Add the method pstricks 43
Allow backslash in command name 48
Declare predefined commands after unicode-math settings . . . 50
Fix esvect font sizes 36
Get the correct rule thickness with unicode-math 45
Remove useless `\AtBeginDocument` 38
Use `\def` instead of `\let` for `\ovar@rightarrow` and `\ovar@leftarrow` 36
Use `\harrowextender`, if availlable. 37
Use boxes in starred variant for better performances 49
v1.3
General: Process options in the order specified by the user . . . 36
Replace esvect conditionnal by the definition of a control sequence 50
Use a control sequence to store the esvect option 35
Use only one `\PassOptionsToPackage` with esvect 36
v1.4
General: Make commmands robust 48
v1.5
General: Make `\overarrowthickness` and `\overarrowsmallerthickness` always available 46
Redefine `\overbar` →P. 22 so that it can be properly nested. 52
Redefine `\underbar` →P. 23 so that it can be properly nested. 52
Save math style 44, 46